

审定成绩：_____

重庆邮电大学
毕业设计（论文）

中文题目	知识融入大语言模型的智能用户故事 生成系统设计与实现
英文题目	Design and Implementation of Intelligent User Story Generation System Integrating Knowledge into Large Language Models
学院名称	计算机科学与技术学院（示范性软件学院）
学生姓名	翁艺恺
专 业	软件工程
班 级	13002205
学 号	2022214200
指导教师	黄江平 讲师
答 辩 组 负 责 人	黄颖 副教授

二〇二六年五月
重庆邮电大学教务处制

计算机科学与技术学院本科毕业设计（论文）诚信承诺书

本人郑重承诺：

我向学院呈交的论文《知识融入大语言模型的智能用户故事生成系统设计与实现》，是本人在指导教师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明并致谢。本人完全意识到本声明的法律结果由本人承担。

年级 2022 级

专业 软件工程

班级 13002205

承诺人签名 翁艺恺

2026 年 5 月 18 日

学位论文版权使用授权书

本人完全了解重庆邮电大学有权保留、使用学位论文纸质版和电子版的规定，即学校有权向国家有关部门或机构送交论文，允许论文被查阅和借阅等。本人授权重庆邮电大学可以公布本学位论文的全部或部分内容，可编入有关数据库或信息系统进行检索、分析或评价，可以采用影印、缩印、扫描或拷贝等复制手段保存、汇编本学位论文。

（注：保密的学位论文在解密后适用本授权书。）

学生签名：翁艺恺

指导老师签名：

日期：2026 年 5 月 18 日

日期：2026 年 5 月 18 日

摘要

随着大语言模型技术的快速发展，面向软件需求工程的智能化辅助逐渐成为学术界和工业界关注的重要方向。用户故事作为敏捷开发中一种简洁且面向价值的需求表达方式，在连接利益相关者意图与软件实现之间发挥着关键作用。然而，传统的用户故事编写仍高度依赖人工整理，容易出现语义模糊、表达不一致、功能覆盖不完整以及可追踪性不足等问题。针对上述问题，本文设计并实现了一种融合大语言模型与轻量级领域知识库的智能用户故事生成系统。

在方法设计上，本系统以自然语言需求为输入，首先通过语义解析完成结构化需求理解，包括角色识别、行为提取、条件识别和目标抽取。在此基础上，引入轻量级领域知识库，对术语消歧、语义补全、异常场景补充和约束注入提供支持，从而提升生成结果的完整性与可控性。随后，系统根据增强后的需求表示，自动生成符合敏捷模板的标准用户故事，并进一步将其拆解为具有依赖关系的可执行开发任务。

在功能实现上，系统提供可视化交互界面，用于展示结构化需求、用户故事、任务清单和依赖关系图，支持用户对生成结果进行查看、审查和导出。此外，系统还具备结果评估和面向代码的辅助生成能力。

实验结果表明，本文所设计的系统在语义补全质量、用户故事生成完整性以及任务拆解合理性等方面均有一定参考意义，能够形成从需求理解到用户故事生成、任务拆解与可视化展示的端到端智能支持流程。

关键词：大语言模型；知识融合；用户故事生成；敏捷开发；需求工程

Abstract

With the rapid development of large language models, intelligent support for software requirements engineering has become an important research topic in both academia and industry. However, traditional user story writing still relies heavily on manual effort, which often leads to ambiguity, inconsistency, incomplete functional coverage, and weak traceability. To address these issues, this study designs and implements an intelligent user story generation system that integrates LLMs with a lightweight domain knowledge base.

In terms of methodology, the proposed system takes natural language requirements as input and first performs structured requirement understanding through semantic analysis, including role identification, action extraction, condition recognition, and goal extraction. On this basis, a lightweight domain knowledge base is introduced to support term disambiguation, semantic completion, exception scenario supplementation, and constraint injection, thereby improving the completeness and controllability of generated results. Subsequently, based on the enhanced requirement representation, the system automatically generates standardized user stories that conform to agile templates and further decomposes them into executable development tasks with dependency relationships.

Regarding system implementation, the system provides a visual interactive interface for displaying structured requirements, user stories, task lists, and dependency graphs, allowing users to view, review, and export generated results. In addition, the system supports result evaluation and code-oriented auxiliary generation capabilities.

Experimental results demonstrate that the proposed system exhibits promising performance in semantic completion quality, user story generation completeness, and task decomposition rationality. It establishes an end-to-end intelligent support workflow covering requirement understanding, user story generation, task decomposition, and visual presentation.

Keywords: Large language model; Knowledge fusion; User story generation; Agile development; Requirement engineering

目录

第 1 章 引言	1
1.1 研究背景及意义	1
1.2 研究目的	2
1.3 国内外研究现状	3
1.3.1 国外研究现状	3
1.3.2 国内研究现状	5
1.3.3 综述点评	7
1.4 研究内容	8
1.5 研究方法	9
1.6 论文结构	10
第 2 章 关键技术介绍	12
2.1 面向智能用户故事生成的领域知识增强方法	12
2.1.1 轻量级领域知识库的构建策略	13
2.1.2 知识增强与检索机制	14
2.1.3 基于提示词工程的“知识+模型”协同融合	15
2.2 结构化输出与结果规范化	17
2.3 自动化结果评估方法	17
2.4 本章小结	18
第 3 章 系统需求分析	19
3.1 需求获取	19
3.2 需求分析	20
3.2.1 业务需求分析	20
3.2.2 用户需求分析	21
3.2.3 功能需求分析	22
3.2.4 非功能需求	23
3.2.5 约束需求分析	24
3.3 需求规约	25
3.4 需求验证	26
3.5 本章小结	28
第 4 章 系统设计	29
4.1 概要设计	29
4.1.1 设计目标	29

4.1.2 系统整体架构	29
4.1.3 技术架构设计	31
4.1.4 系统业务流程设计	32
4.2 详细设计	32
4.2.1 前端设计	32
4.2.2 后端设计	34
4.2.3 需求文本处理模块设计	36
4.2.4 知识增强模块设计	37
4.2.5 大语言模型生成模块设计	39
4.2.6 用户故事生成与结果评估设计	40
4.2.7 代码辅助生成模块设计	41
4.3 本章小结	42
第 5 章 系统实现	43
5.1 系统整体架构实现	43
5.1.1 数据层的初始化	43
5.1.2 核心业务层	44
5.1.3 前端交互层的界面呈现	46
5.2 各功能实现模块	47
5.2.1 自然语言需求生成功能实现	47
5.2.2 代码片段反向生成用户故事功能实现	49
5.2.3 知识增强与结构化结果展示功能实现	50
5.2.4 用户故事与任务拆解展示功能实现	53
5.2.5 任务依赖可视化功能实现	54
5.2.6 结果评估功能实现	56
5.2.7 相关代码辅助生成功能实现	57
5.2.8 知识模块管理与统计模块功能实现	59
5.2.9 导出功能实现	61
5.3 系统集成过程	61
5.4 本章小结	62
第 6 章 系统测试与结果分析	63
6.1 测试环境	63
6.2 测试用例设计	64
6.3 单元测试	64
6.4 集成测试	65
6.5 系统测试	66

6.6 功能测试	67
6.7 性能测试	68
6.8 验收测试	70
6.9 测试结果	71
6.10 测试结论与分析	72
6.11 本章小结	72
第 7 章 结论与展望	73
7.1 研究成果总结	73
7.2 研究局限与未来展望	74
参考文献	75
致谢	81

第 1 章 引言

1.1 研究背景及意义

人工智能技术一路演进，以大语言模型为代表的新一代生成式 AI，在自然语言理解、知识推理还有文本生成这些任务上，表现出了越来越强的能力，这种进步也深刻影响到了软件工程的研究与实践^[1]。在软件开发的全过程里，需求工程这个阶段，一直都决定着系统能不能成功，需求获取得充不充分、需求表达得规不规范、需求分析得准不准确，会直接关系到后面设计、开发、测试和维护这些工作的质量跟成本^[2,3]。正因如此，怎么借助大语言模型把需求处理的效率提上去，已经变成了当前软件工程走向智能化的重要研究方向之一^[4,5]。

在敏捷开发的模式下，用户故事是连接业务目标与技术实现的一个关键载体，它既要能准确地把用户角色、核心需求以及业务价值表达出来，又得为接下来的迭代计划、任务拆解和测试验证提供一个可靠的基础^[6]。不过在实际项目里头，用户故事的编写通常得靠产品经理、需求分析师或者开发人员手工去整理，这个过程很容易受到经验水平的差异、对业务的理解有偏差还有沟通成本太高等因素的干扰，结果就出现了表达模糊、结构不统一、关键信息漏掉，甚至很难直接拿来指导实施这些问题^[7]。软件系统的规模和业务的复杂程度一直在提升，过去那种主要靠人工整理的方式，已经很难再满足快速迭代和高质量交付的新要求了，相关的系统综述也点明了，敏捷需求工程长期面对着需求变动频繁、跨角色沟通成本高，以及知识传递不够充分等挑战，这也正是对用户故事自动化支持需求不断增长的一个重要原因^[8-10]。

近些年，研究者们开始试着把大语言模型应用到需求理解、需求质量检查、用户故事生成和软件过程自动化这些任务里去^{[11][12]}。这给智能化的需求处理提供了新的技术底子。不过现有的一些研究也指出了，通用的这类大模型在碰上复杂的业务场景时，还是容易把前置条件、异常流程和安全约束给忽略掉，生成出来的内容在一致性、可解释性以及可执行性方面，都还存在着一些短板^[13]。为了让模型在专业场景下表现得更可靠，知识增强、检索增强生成，还有知识图谱融合这些方法，已经慢慢变成了重要的研究方向^[14-17]。

基于上述背景，本文将“知识融入大语言模型的智能用户故事生成系统设计与实现”作为研究对象，尝试通过“轻量级领域知识库+大语言模型”的方式，构建面向敏捷开发场景的智能需求处理系统。从理论层面看，本研究有助于拓展大语言模型在需求工程中的应用边界，丰富知识增强生成在软件工程场景下的研究内容^[18]；从实践的层面来看，则有助于把需求表达的规范性、用户故事生成的完整性以及任务拆解的可执行性都提升起来，为敏捷团队提供一个更高效、更容易追踪的需求支持工具。所以说，这篇论文的研究内容，既有一定的学术价值，也具备着相应的工程应用意义。

1.2 研究目的

本文旨在针对敏捷软件开发中需求理解偏差较大、用户故事编写效率较低、任务拆解依赖人工经验以及生成结果缺乏工程可执行性等问题，设计并实现一个融合大语言模型与轻量级领域知识库的智能用户故事生成系统。该系统以自然语言需求为主要输入对象，通过知识增强、结构化生成和结果可视化等方式，形成从需求理解到用户故事生成再到任务拆解与辅助开发输出的完整处理流程。

围绕这一目标，本文主要希望解决以下几个方面的问题。其一，如何利用大语言模型实现对非结构化需求文本的有效理解，从原始需求中提取角色、行为、条件和目标等核心语义信息；其二，如何将领域知识有效引入生成过程，以降低模型在复杂业务场景下出现术语误用、规则遗漏和逻辑偏差的风险；其三，如何使生成结果不仅停留在用户故事文本层面，而是能够进一步支持任务拆解、依赖关系构建、结果评估和代码辅助输出，从而增强系统的工程应用价值^{[19][20]}。

基于此，本文希望构建一套较为完整的技术方案与系统原型，使系统能够在满足学术研究目标的同时，也具备较好的实际可用性。具体而言，系统应能够支持需求输入、知识增强、用户故事生成、任务拆解、依赖关系展示、结果评估与代码辅助生成等功能，并通过前后端一体化实现和系统测试验证其可行性。本文希望通过该研究，为需求工程智能化、用户故事自动生成以及知识增强型大模型应用提供一定的理论参考和实践案例。

1.3 国内外研究现状

1.3.1 国外研究现状

近年来，大语言模型在软件工程领域的应用逐渐从代码生成、缺陷修复扩展到需求工程、需求分析和软件过程自动化等环节。Antaputra 等对人工智能与敏捷需求工程结合研究进行了系统回顾，指出 AI 技术正逐步进入需求获取、需求分析与需求表达辅助等环节，但其工程可信性和场景适配能力仍需进一步验证^[21]。Hou 等对大语言模型在软件工程中的应用进行了系统综述，指出了 LLM 已经在代码生成、软件测试、需求分析、文档生成等任务中展现出较强的通用能力，但在工程场景落地时仍面临可靠性、可解释性和领域适配不足等问题^[22]。这些研究说明，LLM 已经成为需求工程智能化的重要技术基础，但仅依赖通用模型并不能完全解决需求表达不规范、语义缺失和工程可执行性不足等问题。

在自然语言需求结构化方面，国外研究主要关注如何将非结构化需求文本转化为更清晰、可分析、可验证的需求表示。Ferrari 等人提出的 PURE 需求数据集为自然语言需求分析、需求分类以及自动化需求处理研究提供了公开标准语料，在需求工程研究中被广泛采用^[23]。Norheim 和 Rebentisch 探索了利用大语言模型对自然语言需求进行结构化处理的方法，证明 LLM 能够辅助识别需求中的角色、行为、对象和约束条件，从而降低人工分析成本^[24]。Mokos 等提出自然语言系统需求的语义建模与分析方法，通过语义关系抽取和模型化表达提高需求规约的清晰性和可验证性^[25]。这些研究与本文“自然语言需求—结构化需求表示—用户故事生成”的处理链路具有较强关联，为本文设计结构化需求字段、JSON 输出格式以及后续任务推理提供了理论参考。

在用户故事自动生成方面，国外相关研究已经从早期的规则模板方法逐渐转向 NLP、机器学习和大语言模型驱动的方法。Yamani 等构建了 UStAI 数据集，利用多个 LLM 从 AI 系统相关论文摘要中生成用户故事，并采用用户故事质量框架进行评估，说明 LLM 在早期需求获取和用户故事构造中具有一定可行性^[26]。Zhang 等提出基于 LLM Agent 的用户故事质量增强方法，通过多个智能体分别承担理解、生成、评价和改写职责，以提升用户故事的完整性和规范性^[27]。Dente 等提出 Text2Stories 任务，用于评价利益相关者访谈文本与生成用户故事之间的一致性，

强调自动生成结果不能只看语言流畅性，还应关注其是否真实覆盖原始需求^[28]。Abed 等从以人为中心开发视角评估 AI 生成用户故事质量，发现 LLM 生成结果在模板完整性和初稿形成方面具有明显优势，但在业务语境贴合度与约束细化方面仍需要人工审查^[29]。

在任务拆解和推理机制方面，国外研究主要集中于提示词工程、推理链优化和多步骤任务规划。Wei 等提出思维链提示方法，证明通过显式引导模型生成中间推理步骤，可以提升模型在复杂推理任务中的表现^[30]。Yao 等提出 Tree of Thoughts 方法，使语言模型能够在多个推理路径中进行搜索、比较和回溯，从而增强复杂问题求解能力^[31]。Sahoo 等对提示词工程方法进行了系统梳理，指出任务指令、输出格式约束、上下文示例和结构化提示对于提升 LLM 输出稳定性具有重要作用^[32]。Ronanki 等进一步面向需求工程场景提出了提示工程指南，强调角色设定、输出结构约束与领域上下文注入对提升需求分析与故事生成稳定性具有关键作用^[33]。这些研究对本文系统中的提示词模板设计具有直接启发意义：本文在调用 LLM 时，不仅要求模型生成用户故事，还要求其按照固定 JSON 结构输出角色、行为、条件、目标、任务列表和依赖关系，从而降低模型自由生成造成的格式混乱问题。

在知识增强与融入大模型生成方面，RAG 和知识图谱增强是近年来国外研究的重要方向。Borgeaud 等人提出通过外部知识检索增强语言模型生成能力的方法，证明知识检索机制能够有效提升模型在知识密集型任务中的准确性与事实一致性^[34]。Li 等从图结构与大语言模型融合角度总结了图学习和 LLM 结合的发展趋势，说明结构化关系知识能够弥补纯文本生成模型在关系推理方面的不足^[35]。这些研究为本文引入轻量级领域知识库提供了技术依据。随着预训练技术的演进，基于大语言模型的代码生成已成为自动化软件工程的核心课题。Chen 等人对 OpenAI 发布的 Codex 模型进行了系统性评估，证明了模型在解决编程任务上的涌现能力^[36]。Roziere 等人推出的 Code Llama 则通过长文本微调技术，确立了模型在处理复杂工程上下文时的领先地位^[37]。Zan 等人通过综述指出，从自然语言需求到代码（NL2Code）的转化关键在于对需求语义的深层捕捉^[38]。Barke 等通过对程序员使用代码生成模型过程的实证研究发现，开发者通常在仓库上下文、人工审查与交互式修正的循环中使用此类工具，说明代码生成模型的工程价值不仅取决于生成正确率，也取决于与真实开发流程的协同方式^[39]。Vaithilingam 等从可用性视角指

出，开发者虽然普遍认可大语言模型代码工具的提效潜力，但在实际使用中仍需要频繁核查、修正和重构生成结果，这说明代码生成能力与工程可用性并不完全等价^[40]。

在测试辅助评估领域，多智能体协作正逐渐成为主流模式。Wang 等人则对 LLM 在软件测试领域的应用进行了系统梳理，强调了测试驱动开发在智能生成流中的价值^[41]。Li 等从软件测试任务视角系统评估了大语言模型的能力边界，指出模型在测试辅助方面具备潜力，但在复杂逻辑覆盖与结果稳定性上仍存在明显差异^[42]。在评估机制方面，利用更强大的模型作为裁判来评价生成内容的语义质量，进一步证明了利用多维指标评估需求对齐度的有效性。Tong 和 Zhang 提出的 CodeJudge 则将代码生成评价从简单通过率扩展到更细粒度的质量判断，为自动化评价生成结果提供了更适合工程场景的实现思路^[43]。

1.3.2 国内研究现状

国内关于大语言模型、知识增强和软件需求质量的研究也在快速发展。陈慧敏、刘知远、孙茂松从社会影响和技术演进角度分析了大语言模型时代的机遇与挑战，指出大语言模型虽然具有较强的文本理解和生成能力，但在可信性、安全性、价值对齐和专业领域应用中仍需要进一步研究^[44]。王昀等总结了大语言模型的发展历程及其在政务领域的应用，认为行业级大模型应用必须结合领域知识、业务流程和安全约束，才能在真实场景中发挥稳定作用^[45]。孙毅等对自然语言预训练模型知识增强方法进行了综述，将知识增强方法按照显式知识、隐式知识和任务适配方式进行归纳，说明外部知识对于提升模型语义理解和推理能力具有重要价值^[46]。这些研究为本文“LLM+轻量级知识库”的总体思路提供了国内理论支撑。

在知识图谱与大语言模型融合方面，张鹤译等研究了大语言模型融合知识图谱的问答系统，指出知识图谱能够提供结构化事实和关系约束，有助于提升大模型生成内容的准确性和可解释性^[47]。冯志伟围绕大模型时代的检索增强技术进行了分析，认为检索增强能够在推理阶段以非参数化方式引入外部知识，缓解模型知识过时、长尾知识不足和幻觉等问题^[48]。王亮进一步从知识服务角度总结了 RAG 的原理、范式与评估方法，指出 RAG 的核心在于将外部知识库检索结果与生成模型结合，使生成结果更加符合特定领域语境^[49]。这些成果说明，在面向专业业务

场景时，仅依靠通用大模型进行文本生成是不充分的，结合知识库或知识图谱进行语义约束与事实补充，已经成为提升模型可靠性的常见技术路线。本文所构建的知识库虽然规模较小，但围绕软件需求场景建立模块化规则，能够为用户故事生成提供前置条件、异常场景、安全约束和典型任务模板，与国内知识增强研究方向保持一致。

在敏捷开发和用户故事质量研究方面，国内已有学者关注用户故事在需求工程中的规范化表达和缺陷检测。王春晖、金芝等提出了一种用户故事需求质量提升方法，通过构建用户故事概念模型和质量准则，检测用户故事中存在的缺失、歧义、重复和冲突等问题，为自动化用户故事质量评价提供了重要参考^[50]。荣国平等对嵌入式系统开发中敏捷方法的应用进行了综述，指出敏捷开发能够通过迭代、反馈和用户参与提升软件开发适应性，但在复杂系统中仍需要加强需求表达、过程管理和质量控制^[51]。项目管理协会和敏捷联盟发布的《敏捷实践指南》系统阐述了敏捷方法的价值、原则和实践方式，强调项目团队应根据不同场景灵活采用敏捷工具和技术，以提升交付效果^[52]。这些研究表明，用户故事并不是简单的自然语言描述，而是敏捷需求工程中的核心载体，其质量直接影响后续开发任务拆解、测试验证和迭代计划。

总体来看，国内研究在大语言模型知识增强、知识图谱问答、RAG 应用和用户故事质量评价方面已经形成了一定基础，但将这些研究成果直接融合到“自然语言需求解析—用户故事生成—任务拆解—依赖关系可视化”的完整系统中的工作仍相对不足。与此同时，国内软件开发实践中大量需求仍以自然语言、会议纪要或简单功能描述形式存在，人工整理用户故事和开发任务不仅效率较低，而且容易出现遗漏、重复和粒度不一致等问题。因此，本文结合任务书要求，设计并实现一个知识融入大语言模型的智能用户故事生成系统：一方面利用 LLM 的语义理解和生成能力完成需求解析、用户故事生成与任务推理；另一方面通过轻量级知识库提供术语消歧、条件补全、异常场景和任务模板约束；最后通过 Web 可视化界面展示结构化需求、用户故事列表、任务清单和依赖关系图，从而提高用户故事理解、需求组织与敏捷开发前期需求处理的效率、规范性和可审查性^[53]。

1.3.3 综述点评

综合国内外研究来看，用户故事自动生成已经从传统模板填充阶段发展到大模型驱动的智能生成阶段，但仍存在三个方面的问题，一是现有研究大多把目光放在“能不能生成用户故事”上，对用户故事与原始需求之间的一致性、完整性和工程可执行性关注得不够，二是大语言模型虽然能产出流畅的文本，在复杂业务场景下却很容易漏掉前置条件、异常路径和安全约束，导致生成的结果很难直接用到开发里，三是不少研究习惯于把用户故事生成跟任务拆解、依赖关系分析、可视化展示割裂开，缺少一个面向敏捷开发流程的完整系统。在质量研究方面，Ronanki 等系统探索过用 ChatGPT 评估用户故事质量的可行性与可信度，发现 LLM 按照 INVEST 准则给出的评级和人工评估有着较高一致性，还提出了“三选一最优”策略来提升结果的稳定性^[54]。Santos 等则直接追问“AI 生成的用户故事到底够不够好”，指出它在可读性和早期构思支持上表现还行，但在验收标准、边界条件与可执行性上仍有欠缺^[55]。

针对这些不足，本文的研究重点远不止生成用户故事文本，更是要构建一个从自然语言需求到结构化需求、从用户故事到任务清单、再从任务清单到依赖关系图的完整系统，通过轻量级知识库去约束和增强大语言模型，让模型在生成过程中能结合常见软件模块的前置条件、异常场景、典型任务和安全约束，把纯模型生成的不确定性给降下来；同时，系统要求模型输出统一的 JSON 结构，并在前端用故事卡片、任务表格和依赖图进行展示，让生成的内容有了可查看、可修改、可导出和可评价这些工程特征。因此，本文的创新点可以概括为几个方面，一是提出了一种面向敏捷需求场景的轻量级知识增强用户故事生成框架，通过把领域知识规则融入大语言模型生成过程，提升了用户故事生成的完整性；二是设计了一套基于结构化需求表示的任务自动拆解机制，做到了用户故事到开发任务的自动映射；三是构建了一个集需求理解、故事生成、任务依赖分析和结果可视化于一体的端到端系统原型。

1.4 研究内容

本文主要探讨基于大语言模型（LLM）以及轻量级领域知识库相结合的智能用户故事生成系统的研究工作，以解决传统敏捷开发中存在的需求理解偏差大、用户故事撰写效率低、任务拆解主观性强等问题。随着软件工程逐渐向智能化和自动化方向发展，开发人员对高效需求工程工具的需求不断增加。因此，本文以自然语言的需求作为输入，利用大语言模型强大的语义理解和生成能力，结合结构化的知识库进行术语消歧和上下文补全，将模糊、非结构化的需求转化为清晰、可操作的需求。这不仅可以提高需求分析的准确度，而且有利于后续自动化生成。

研究内容主要包括需求理解、知识增强、用户故事生成、任务拆解与依赖分析以及结果可视化等模块，其整体流程如图 1.1 所示。其中，首先对原始需求进行结构化处理，再结合知识库完成语义增强，随后生成用户故事和开发任务，最终输出依赖关系图和结果评估信息。

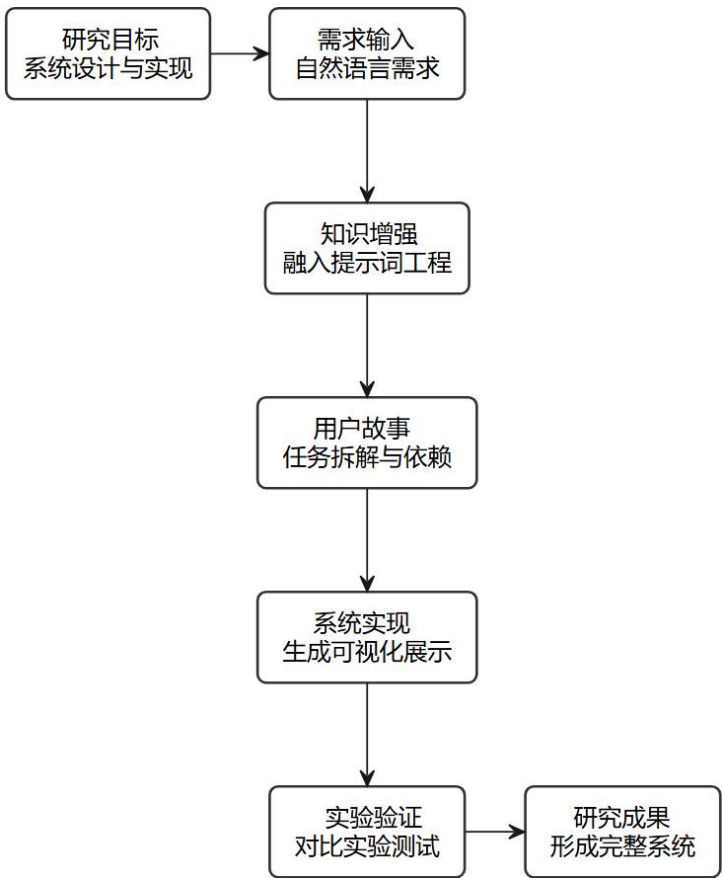


图 1.1 研究内容图

基于结构化需求，系统根据标准敏捷模板自动生成符合“作为一个……希望……以便……”格式用户故事，然后调用知识库中任务模式以及业务规则，对每一个故事进行细粒度功能点划分。经过多次知识增强推理过程，系统可以自动发现任务之间依赖关系、推断出合理执行顺序并且预测可能出现问题情况，从而形成完整任务链以及执行逻辑图谱。整个流程以图形化方式展示用户故事、任务列表以及其关系，允许用户进行交互式查看和修改。本文通过查阅相关文献了解技术范围，在此基础上进行实验比较不同提示词策略以及知识库结构对生成效果影响，同时与纯 LLM 基线方法进行比较。评价标准包括语义补全准确率、故事完整性得分以及任务推理逻辑一致性等。通过参考软件工程 Prompt Pattern 方面的研究，通过角色约束、输出模板约束以及任务依赖关系约束，提高大语言模型输出结果的一致性与可解析性^[56]，提示词模式目录与仓库级检索策略^[57,58]，为本文利用轻量级知识库提高生成质量提供了实践指南。

1.5 研究方法

本文采用文献研究、系统设计、实验验证及对比分析等方法开展研究工作，保证系统设计合理性和实用性。利用系统文献综述的方法，总结近五年来国内外对于大语言模型、知识融合、用户故事生成以及敏捷需求工程等相关方面的研究情况以及所面临的问题。查阅了包括 IEEE Transactions on Software Engineering、ACM Transactions on Software Engineering and Methodology 等期刊上有关 LLM 用于软件工程任务的文章以及 GitHub、arXiv 等开源网站上相关项目，总结出目前智能需求处理系统中存在的问题主要是在语义理解上不够深入，在知识引导方面存在不足以及任务推理不够严谨等问题。然后根据以上问题提出本文的研究思路：建立一个轻量级领域知识库，与 LLM 的语义生成能力相结合，减少对模型本身泛化能力的过分依赖而造成术语使用不当或者逻辑不通等问题。这一部分是后面系统架构设计和主要模块划分的基础。

本文以实验验证以及对比分析的方法进行系统的实现以及改进工作。即设置多个对照实验，比较 LLM+规则模板、LLM+知识库增强两种方式在同一需求语料上所生成用户故事的质量。评价标准包括语义完整性（是否包含角色、目标、价值三要素）、术语准确性（是否正确使用领域词汇）、任务可拆解性（是否具有

明确功能点）以及逻辑一致性（是否存在矛盾或者重复）。另外，还加入了性能测试，记录系统的平均响应时间、内存使用情况以及错误率，保证其在真实开发环境中的实用性。不断改进提示词工程、优化知识库实体关系结构以及调整任务拆解规则来提高系统的鲁棒性和专业性。整个研究工作围绕“设计—实现—评估—优化”的循环进行，最终得到一套从需求理解到故事生成再到任务推理以及可视化交互的需求工程系统，满足敏捷开发对于快速、准确、可追踪的需求文档的需求。Du 等在类级代码生成场景下对多种大语言模型进行了系统比较，结果表明上下文完整度、结构约束和任务粒度会显著影响生成代码的正确性与可维护性^[59]，Yeo 面向代码生成能力的系统评价框架，从功能正确性、结构合理性和任务完成度等角度为不同模型的工程表现比较提供了统一依据^[60]，为本文后续代码辅助生成功能的实现与评价提供了参考。本文不仅进行理论研究，还实现了一个可运行的智能用户故事生成系统，并通过真实需求数据进行测试验证。

1.6 论文结构

整篇论文由七个章节组成，论文章节结构如图 1.2 所示

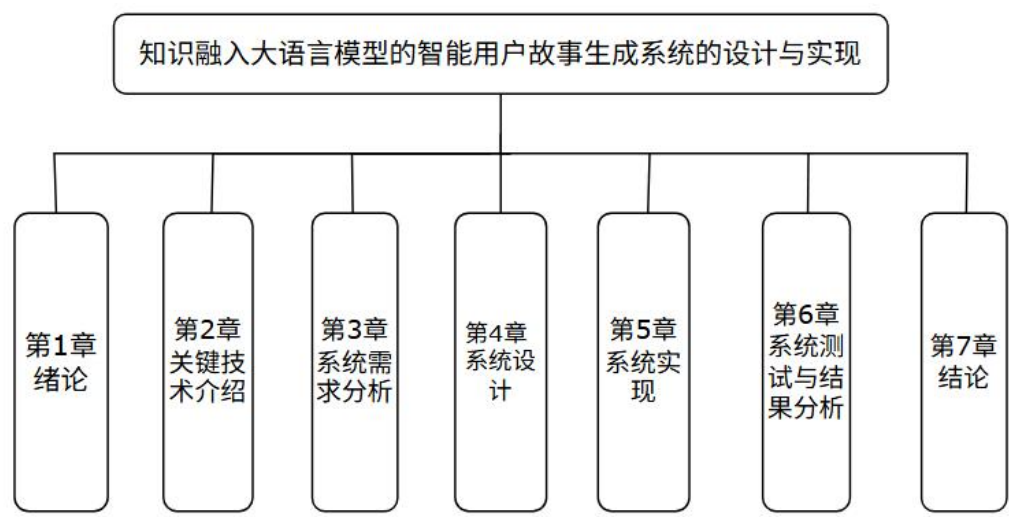


图 1.2 论文章节结构图

第 1 章为引言，主要介绍本文的研究背景与研究意义，梳理国内外相关研究现状，明确本文的研究内容、研究方法以及整体技术路线。

第 2 章为关键技术介绍。该章围绕智能用户故事生成场景所依赖的核心技术展开，重点介绍大语言模型在需求工程中的应用特点、轻量级领域知识库的构建思路、知识增强与检索机制，以及基于提示词工程的“知识+模型”协同方法，为后续系统需求分析、系统设计与实现提供技术支撑。

第 3 章为系统需求分析。该章从需求获取、需求分析、需求规约和需求验证四个方面展开，围绕业务需求、用户需求、功能需求、非功能需求及约束需求，对智能用户故事生成系统的建设目标和功能边界进行系统论证，为后续系统设计与实现提供依据。

第 4 章为系统设计。该章从系统整体架构、技术架构、业务流程以及各功能模块设计等方面，对系统的总体方案和模块划分进行说明，明确系统各组成部分之间的关系及运行逻辑。

第 5 章为系统实现。该章结合具体代码与运行环境，对数据层、核心业务层和前端交互层的实现过程进行介绍，并对主要功能模块和系统集成过程进行展示，说明系统由设计到落地的实现路径。

第 6 章为系统测试与结果分析。该章在给定测试环境下，对系统主要功能进行验证，并结合测试结果对系统生成效果、运行表现和整体可用性进行分析，以检验系统是否满足预期需求。

第 7 章为结论与展望。该章对全文研究工作进行总结，归纳本文的主要成果与创新点，分析当前研究存在的不足，并对后续可进一步开展的研究方向进行展望。

第2章 关键技术介绍

2.1 面向智能用户故事生成的领域知识增强方法

大语言模型有比较强的自然语言理解与生成能力，在需求工程领域体现出了不低的应用价值，对本系统来说，它能发挥的作用，一是去理解用户输入的自然语言需求，把里面的角色、行为、条件、目标等关键信息识别出来，二是基于这些结构化语义，生成出符合敏捷开发规范的用户故事文本，三是进一步将用户故事拆解为前端、后端、数据库等层面的工程任务，这些能力与第3章提出的功能需求高度契合，因此它成了本系统智能处理能力的核心技术基础；不过，大模型在软件需求工程中的应用也有局限，一个方面，模型虽具备语言生成能力，但对特定业务场景的规则掌握得并不稳定，容易忽略前置条件、异常处理和安全约束这些隐性知识，另一个方面，模型在自由生成时可能出现输出格式不稳定、术语不统一、任务依赖关系不合理等情况，这与需求工程对规范性和可执行性的要求存在一些矛盾；所以，本系统没有将原始需求直接交给模型自由生成，而是引入了领域知识和结构化约束机制，一个做法是通过知识库补充领域规则，降低模型遗漏关键业务信息的风险，另一个做法是利用提示词工程和结果规范化处理，提高模型输出的稳定性与可控性，由此能看出，它在本系统中并非孤立的文本生成工具，而是与知识库、结构化约束一起，共同构成了智能需求分析方法体系的重要部分。大语言模型在需求工程中的适用性分析如表2.1所示。

表 2.1 LLM 在需求工程适用分析表

方面	体现内容	对本系统的意义
自然语言理解能力	能理解非结构化需求文本	支持角色、行为、条件、目标提取
文本生成能力	能生成自然语言故事与任务描述	支持用户故事和任务清单输出
语义归纳能力	能从代码中反推业务需求	支持代码片段生成用户故事
局限一	容易遗漏领域规则	需要知识增强机制补充
局限二	输出格式不稳定	需要结构化提示词约束
局限三	可能出现依赖关系不合理	需要后处理规范化机制

2.1.1 轻量级领域知识库的构建策略

针对系统业务需求与约束，系统不能仅依赖通用大语言模型完成用户故事生成，还需要引入能够反映典型业务规则的领域知识支撑。为此，本文构建了轻量级领域知识库，用于存储常见软件业务模块的别名、必选要素、前置条件、异常场景、典型任务和安全约束等信息，为知识增强生成提供基础。

之所以采用“轻量级”知识库，而非复杂知识图谱或大型知识管理平台，主要基于两点考虑。其一，本系统面向毕业设计和本地演示场景，要求部署简单、运行成本低、维护方便；其二，本系统当前聚焦的是“需求增强”而非大规模推理，因此知识库更强调结构清晰、便于检索和易于扩展，而不是追求复杂推理能力。基于这一定位，轻量级领域知识库更符合本系统的实际需求。

在系统早期阶段，领域知识以静态文件形式进行组织。这种方式实现简单，适合系统初始开发阶段快速验证知识增强，随着知识模块增加，其缺点逐渐显现，不便于增删改查、难以按条件检索、无法记录更新时间，也不利于系统后续扩展。

为了提升知识管理能力，系统将领域知识从静态文件组织方式升级为轻量级数据库存储方式。与早期静态存储相比，数据库方案在可管理性、可扩展性和可查询性方面具有明显优势。还能方便后续新增和维护，系统能够基于模块名称和别名等信息进行快速检索，为知识增强过程提供支持。

此外，数据库方案更符合提出的约束需求，即系统需要在轻量化部署场景下稳定运行。采用轻量级数据库能够在不显著增加复杂度的前提下，提高知识管理效率，因此成为本系统领域知识组织的合适方案。知识库对比方案如表 2.2 所示。

表 2.2 知识库对比方案表

方案	优点	缺点	适用阶段
静态文件存储	实现简单，原型验证快	不便维护，查询能力弱，扩展性差	早期原型阶段
轻量级数据库存储	结构清晰，便于查询和扩展，支持长期维护	需要额外设计存储结构	系统正式实现阶段

为了使知识能够被系统有效检索和利用，领域知识不能仅以自然语言说明的形式存在，而需要采用统一的数据组织方式进行表示。结合本系统特点，知识模

块以“模块名称+模块类别+别名+规则信息”的方式组织，其中规则信息进一步细化为必选要素、前置条件、异常场景、典型任务和安全约束等部分。这样的组织方式既能够反映软件业务模块的核心特征，又便于在生成阶段被模型直接利用。

在知识表示层面，模块名称用于标识业务主题，别名用于支持不同表达方式下的匹配，必选要素用于补充该模块必须具备的核心内容，前置条件用于约束功能触发前提，异常场景用于提示系统补充异常处理逻辑，典型任务用于为任务拆解提供参考，安全约束则用于提醒模型在生成结果时保留必要的业务合规信息。知识模块内容构成说明如表 2.3 所示。

表 2.3 知识模块内容表

知识要素	含义	在系统中的作用
模块名称	业务模块的核心标识	用于主题识别
别名	模块的同义表达	用于提升匹配召回率
必选要素	功能必须包含的关键内容	用于补充需求要点
前置条件	功能触发前提	用于约束业务逻辑
异常场景	常见失败或异常情况	用于增强健壮性分析
典型任务	模块对应的常见开发任务	用于指导任务拆解
安全约束	需要满足的安全规则	用于提高生成结果合规性

2.1.2 知识增强与检索机制

知识增强与检索机制是连接领域知识库与大语言模型的关键环节，也是本系统实现高质量生成的重要方法基础。当用户输入一段自然语言需求后，系统并不是直接将其送入模型，而是先对输入文本进行归一化处理，减少表达差异对匹配结果带来的影响。随后，系统根据模块名称及其别名在知识库中进行匹配，筛选出与当前需求最相关的知识模块。

在检索到目标模块后，系统会从知识库中提取该模块对应的必选要素、前置条件、异常场景、典型任务和安全约束，并将这些内容组织为统一的知识增强上下文。该上下文并非最终输出结果，而是作为后续模型生成阶段的重要输入，用于约束模型关注更贴近实际业务的关键内容。

如果当前需求未命中具体知识模块，系统也不会中断处理流程，而是返回通用的软件工程提示信息，保证系统在不同输入场景下仍然具备可用性。由此可见，知识增强机制本质上是在大语言模型生成之前建立一层业务先验过滤与补充机制，以降低模型忽略关键规则或产生自由幻觉的风险。

这一机制对应了第3章中提出的业务需求、功能需求和非功能需求。一方面，它提升了生成结果的完整性和可解释性；另一方面，也增强了系统在复杂业务场景下的稳定性和可用性。

2.1.3 基于提示词工程的“知识+模型”协同融合

在获取到原始需求文本和知识增强上下文之后，系统需要通过合理的提示词工程，将知识库与大语言模型有机结合起来。提示词工程的目标不是简单要求模型“生成结果”，而是通过角色设定、任务约束、输出格式约束和异常处理约束，使模型在可控条件下完成需求理解和用户故事生成。

首先，在角色设定层面，系统需要明确模型在当前任务中扮演的是敏捷开发与需求工程专家，从而让模型在生成时优先采用软件工程视角理解需求。其次，在任务约束层面，系统需要明确要求模型同时完成结构化需求提取、用户故事生成和任务拆解，避免模型只输出单一文本结果。再次，在输出格式层面，系统需要限制模型必须返回结构化 JSON 结果，并明确字段组成和取值范围，例如任务类型限定为前端、后端和数据库三类，依赖关系字段必须使用标准列表形式表达。

在此基础上，知识增强上下文作为提示词的一部分被嵌入到模型输入中，使模型在生成过程中能够主动参考前置条件、异常场景和安全约束等领域规则。这种“知识+模型”的协同方式兼顾了知识库的稳定性和模型的语义推理能力，是本系统区别于纯大模型文本生成系统的核心方法。

需要指出的是，提示词工程并不能完全替代后处理机制。因此，在模型生成完成后，系统还需要通过结果规范化方法，对字段类型、任务编号、依赖关系等进行进一步清洗和校正。提示词约束与后处理规范化共同构成了本系统结构化生成策略的两级保障机制。如图 2.1 所示，系统首先对用户输入需求进行预处理和知识检索，然后构建知识增强上下文，通过提示词模板与大语言模型协同生成结构化结果，最后经过结果规范化处理输出最终结果。

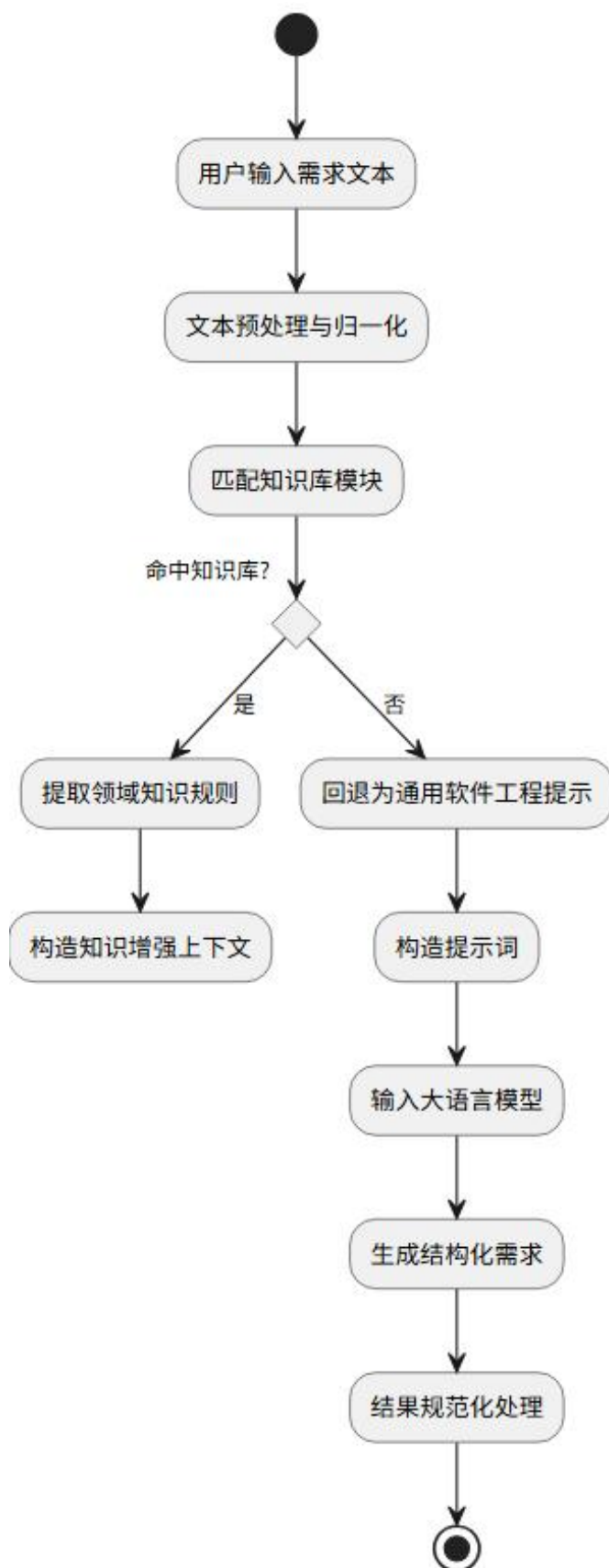


图 2.1 “知识+模型”生成总体流程图

提示词工程关键约束如表 2.4 所示。

表 2.4 提示词工程约束表

约束类型	主要内容	作用
角色约束	明确模型扮演需求分析与敏捷开发专家	提高生成语境一致性
任务约束	同时输出结构化需求、用户故事和任务清单	避免输出内容残缺
模板约束	用户故事必须遵循统一模板	保证表达规范性
格式约束	输出必须为结构化 JSON	便于后续解析与展示
类型约束	任务类型限定为前端、后端、数据库	保持任务分类统一
依赖约束	depends_on 字段必须表示前置任务编号	支持依赖关系构建

2.2 结构化输出与结果规范化

为了让大语言模型生成的结果能稳定地用于前端展示、自动评估和后续任务推理，本整体在生成阶段使用了‘结构化输出约束+结果规范化处理’的双层控制方式。和自由文本生成不一样，整体不会直接接收模型返回的自然语言描述，而是让模型按照统一的 JSON 结构输出结果。核心字段有三部分，分别是结构化需求、用户故事列表和任务清单。其中，结构化需求包含角色、行为、条件和目标这四类语义信息。任务清单则要求包含任务编号、任务名称、任务类型、所属用户故事以及前置依赖关系等关键内容。

模型生成完成之后，系统还要对输出结果做进一步的规范化处理。一种是把角色、行为、条件、目标等字段统一转换成列表结构，防止因为模型输出格式不一样，影响后续解析。二，对任务信息进行标准化处理，包括补全任务编号、统一任务类型取值范围、清洗空字段，还有规范 depends_on 依赖列表的表达形式。对于不满足约束条件的异常结果，系统还会返回统一的默认结构，保证前端页面、评估模块和导出功能在异常情况下也能稳定运转。

结构化输出方式解决的是“模型必须按什么格式生成”的问题，结果规范化方式解决的是“整体如何稳定接收和使用生成结果”的问题。这两种方式相互配合，一起构成了本整体结构化生成能力的重要保障，也为后续任务拆解展示、依赖关系形成和结果质量评估打下了数据基础。

2.3 自动化结果评估方法

在智能用户故事生成系统里，仅仅能够生成结果，不能说明系统有实际应用价值，还得回答“生成结果质量怎么样”“是否符合需求工程的可用性要求”等问题。所以，本文在系统中设计了自动化结果评估方法，可以从结构化完整度、用户故事质量、任务拆解质量和依赖关系质量这四个方面，对生成结果进行综合评价。

在结构化完整度方面，系统会重点去检查核心语义要素，像角色、行为、条件和目标这些，有没有被有效提取出来。在用户故事质量方面，主要是判断生成的内容是不是符合“作为一个.....我想要.....以便.....”这个标准表达模板。在任务拆解质量方面，会关注任务数量合不合理、任务类型划分规不规范，还有是不是具备基本的工程可执行性。在依赖关系质量方面，重点是验证任务有没有包含规范的前置依赖字段，以及任务之间有没有基本的顺序逻辑。系统会按照上面这些规则，对各个方面分别进行赋分，然后计算出综合得分，同时还会生成对应的结果总结和改进建议。

这种评估方法有实现简单、结果能复现和计算成本低等特点，适合本整体目前的毕业设计和原型验证场景。它可以在用户完成生成后马上给出质量反馈，帮助用户判断当前结果能不能直接用。它也为后续实验对比和整体测试提供了统一的评价依据，让整体从“能够生成”进一步提高到“能够评判生成质量”。

2.4 本章小结

本章分析了智能用户故事生成整体实现过程中涉及的重要技术。一种是，介绍了面向智能用户故事生成的领域知识增强办法，说明了轻量级领域知识库、知识增强检索方式和提示词工程在提高生成完整性和可控性方面的重要作用。二，阐述了结构化输出和结果规范化办法，说明系统怎样保证模型输出结果能被稳定解析和使用。第三，分析了自动化结果评估办法，明确了系统对生成质量进行多维评价的基本思路。这些重要技术一起形成了本系统从输入理解、知识补充到结果生成和质量评估的技术基础。最后，分析了自动化结果评估方法，明确了系统对生成质量进行多维评价的基本思路。上述关键技术共同构成了本系统从输入理解、知识补充到结果生成与质量评估的技术基础。

第3章 系统需求分析

3.1 需求获取

本系统面向智能用户故事生成场景，研究目标是构建一套能够完成需求理解、用户故事生成、任务拆解、结果评估与代码辅助输出的智能化软件工具。为了保证系统需求来源清晰、内容完整且具有工程可行性，本文从任务目标、敏捷开发实际痛点、典型用户使用场景以及现有技术实现条件四个方面开展需求获取工作。

首先，从目标出发，系统需要完成“知识融入大语言模型的用户故事生成”这一核心任务，因此需求获取必须围绕知识增强、用户故事生成、任务拆解和可视化展示等关键能力展开。其次，从软件工程实践场景出发，传统用户故事编写通常依赖人工经验，容易出现需求表达模糊、业务规则遗漏、任务拆解粗糙以及依赖关系不清晰等问题，因此系统需要具备较强的自动化分析和规范化输出能力。再次，从目标用户角度出发，产品经理、需求分析人员和开发人员在使用系统时关注的重点并不相同，需求获取需要同时覆盖输入便利性、结果可读性、工程可执行性和输出可留档性。最后，从技术实现条件出发，系统当前采用前后端分离架构，后端基于 FastAPI，前端基于 Vue，知识库采用 SQLite 存储，并通过大语言模型接口完成语义推理，因此需求获取还需充分考虑现有技术路线下的实现边界。

综合上述分析，本系统的需求获取方法主要包括文献调研、场景分析、用户角色分析、同类系统功能归纳以及基于现有实现原型的反向抽取。通过这些方式，可以较为完整地形成系统的业务需求、用户需求、功能需求、非功能需求和约束需求，为后续需求分析与规约提供依据。需求获取来源与方法如表 3.1 所示。

表 3.1 需求获取来源与方法表

需求来源	获取方式	主要形成内容
毕业设计任务目标	课题目标分解	明确系统必须完成用户故事生成与知识增强
敏捷开发实践痛点	场景分析与问题归纳	明确需求模糊、拆解粗糙、依赖不清等问题
用户角色使用场景	角色分析与用例梳理	明确产品经理、需求分析人员等的使用需求
同类系统与相关研究	文献调研与功能归纳	明确智能需求分析系统的常见功能边界
当前系统原型与实现条件	原型分析与技术可行性分析	明确系统应支持的功能及技术约束

3.2 需求分析

在完成需求获取后，需要进一步对需求进行分类和细化。结合本系统的研究目标与实现内容，需求分析主要从业务需求、用户需求、功能需求、非功能需求和约束需求五个方面展开。这样的划分既符合软件工程需求分析方法，也有利于后续将需求转换为可实现、可验证、可追踪的规约条目。

3.2.1 业务需求分析

业务需求强调系统建设的根本目标，即系统为什么要被开发、要解决什么问题。从业务层面看，本系统的建设目标主要体现在以下几个方面。

1) 系统需要解决传统用户故事编写依赖人工经验、表达方式不统一的问题。实际需求分析过程中，不同人员对同一业务需求的理解可能存在偏差，导致生成的用户故事质量不稳定，因此系统应能够将非结构化需求自动转化为规范化用户故事表达。

2) 系统需要解决仅依赖大语言模型生成时可能出现的领域规则遗漏问题。对于登录、支付、审批、订单管理等典型业务场景，如果不引入领域知识，模型可能忽略前置条件、异常场景和安全约束。因此，系统应能够在生成前自动匹配知识库，并将知识上下文融入生成过程。

3) 系统不仅要输出用户故事文本，还需要进一步支持任务拆解和依赖关系表达。对于软件开发团队而言，用户故事只有转化为前端、后端、数据库等可执行任务后，才能真正服务于项目实施。因此，系统需要具备从用户故事到工程任务的延伸能力。

4) 系统还需要支持结果评估与辅助开发能力。生成结果除了“能生成”，还应当“可判断质量”“可进一步用于开发”，因此系统需要能够对结果进行评估，并支持根据选中的用户故事继续生成代码骨架。

综合来看，本系统的业务需求可以概括为：构建一个面向敏捷开发场景的智能需求分析与用户故事生成平台，实现从原始需求输入到用户故事、任务拆解、依赖展示、质量评估和代码辅助输出的完整闭环。系统业务需求如表 3.2 所示。

表 3.2 系统业务需求表

编号	业务需求	说明
BR1	规范化需求表达	将自然语言需求转化为标准用户故事
BR2	提升生成完整性	通过知识增强减少关键信息遗漏
BR3	支持任务落地	将用户故事拆解为可执行开发任务
BR4	支持结果分析	对生成结果进行评分与建议输出
BR5	支持工程辅助	基于用户故事继续生成相关代码骨架

3.2.2 用户需求分析

用户需求强调系统从使用者角度需要提供什么能力。结合系统定位，本系统的主要用户可以概括为需求分析人员、开发人员以及系统维护人员。

对于需求分析人员而言，他们希望系统能够快速接收自然语言需求描述，并自动生成结构化需求结果、标准用户故事和任务清单，从而减少手工整理和反复沟通的成本。对于开发人员而言，他们更关注任务是否已经拆解到可执行层面，是否能够区分前端、后端和数据库任务，以及任务之间是否已经给出清晰依赖关系。对于系统维护人员而言，则希望系统具备知识模块可扩展能力，便于后续根据不同业务场景持续补充知识库。系统主要用例如图 3.1 所示

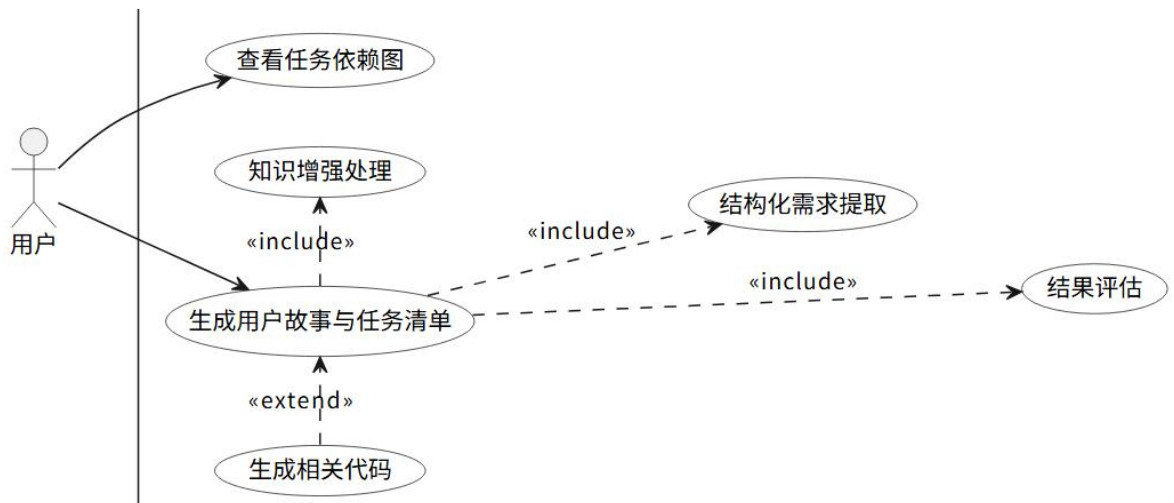


图 3.1 系统主要用例图

3.2.3 功能需求分析

功能需求描述系统应当提供的具体功能，是产品设计和开发中非常核心的部分，对于本系统开发有重要意义。结合系统当前实现目标，系统的功能需求主要包括以下几个方面：

1）系统应支持自然语言需求输入与校验。用户输入业务需求后，系统应能够完成基本合法性检查，并将需求发送至后端进行后续处理。

2）系统应支持代码片段输入与需求反推。当用户提供已有代码片段时，系统应能够从代码中提取业务意图，并进一步生成对应的用户故事和任务结果。

3）系统应支持知识增强上下文生成与展示。系统应根据需求内容匹配知识库模块，自动提取必选要素、前置条件、异常场景、典型任务和安全约束，并将这些内容以增强上下文的形式展示给用户。

4）系统应支持结构化需求提取与标准用户故事生成。系统应能够从原始需求中提取角色、行为、条件和目标四元组，并将结果转换为规范的用户故事文本，方便提示词工程生成对应用户故事。

5）系统应支持任务拆解与依赖关系构建。系统应将用户故事进一步拆解为前端、后端和数据库三类任务，同时给出任务的前置依赖信息，供后续图形化展示使用。

6）系统应支持结果评估功能。系统应从结构化完整度、用户故事质量、任务拆解质量和依赖关系质量等维度对生成结果进行评价，并给出综合得分与改进建议。

7）系统应支持代码辅助生成功能。用户选中某条用户故事后，系统应能够生成与之相关的前端代码、后端代码、数据库 SQL 和项目结构建议。

8）系统应支持结果导出功能。用户应能够将当前页面中的知识增强上下文、结构化需求、用户故事、任务清单、评估结果和代码生成结果导出为 Markdown 文档。

9）系统应支持知识模块管理功能。系统应具备知识模块查询与新增能力，为后续知识库扩展提供支持。

根据以上功能需求，得出功能需求表如表 3.3 所示

表 3.3 功能需求表

编号	功能需求	主要说明
FR1	自然语言需求输入	支持输入、清空与提交自然语言需求
FR2	代码片段输入	支持输入代码并反向提取业务需求
FR3	知识增强预览	生成并展示知识增强上下文
FR4	结构化需求提取	输出角色、行为、条件和目标
FR5	用户故事生成	生成标准模板化用户故事
FR6	任务拆解	生成前端、后端、数据库任务
FR7	依赖关系构建	生成任务前置依赖并支持可视化
FR8	结果评估	输出得分、结论与改进建议
FR9	代码辅助生成	生成项目结构、前后端代码和 SQL
FR10	结果导出	导出 Markdown 格式结果
FR11	知识模块维护	支持知识模块查询与新增

3.2.4 非功能需求

非功能需求强调整体在质量属性方面要达到的要求。对这个整体来说，非功能需求主要包括可用性、可靠性、可维护性、可扩展性、可移植性和安全性这几个方面。

在可用性方面，系统界面要简洁直观，可以让用户用较少步骤完成需求输入、查看结果和导出操作。像知识增强上下文、结构化需求、用户故事、任务清单和评估信息这些重要结果，应该有良好的可读性。

在可靠性方面，整体要对空输入、模型异常、返回格式错误等情况进行处理，防止因为单点异常造成前端页面崩溃。后端要尽量保持统一的响应结构，前端要有相应的容错能力。

在可维护性方面，系统可以使用模块化的方式来达到目标。前端页面、故事列表等要作为相对独立的组件来组织。后端需要把知识增强、用户故事生成等功能分离出来，这样方便后续进行扩展和修改。

在可扩展性方面，整体要允许继续增加知识模块类别、对提示词内容进行优化，并且补充更多接口功能，这样可以适应不同领域的需求。

在可移植性方面，整体要同时支持本地开发部署和容器化部署，这样方便进行答辩演示和后续迁移。

在安全性上，整体要防止在代码里硬编码大模型接口密钥，可以通过环境变量的方式来提供相关配置。对于知识库和生成结果，也要保证数据结构是明确的，调用边界很清晰。

结合以上非功能需求，得到非功能需求表如表 3.4 所示。

表 3.4 非功能需求表

编号	需求类型	需求内容	说明
NFR1	可用性	页面操作简单、结果展示清晰	满足教学与演示场景使用
NFR2	可靠性	异常情况下保持统一返回与页面稳定	避免系统崩溃
NFR3	可维护性	前后端模块解耦，功能职责清晰	便于后续修改与扩展
NFR4	可扩展性	支持知识库、提示词和功能模块扩展	满足后续研究需要
NFR5	可移植性	支持本地部署和容器化部署	便于答辩演示和迁移
NFR6	安全性	接口密钥通过环境变量管理	避免敏感信息硬编码

3.2.5 约束需求分析

约束需求强调整体在开发和运转过程中一定要遵守的技术边界和达到的条件。和功能需求不一样，约束需求不是直接描述整体“要做什么”，而是规定整体“必须在什么条件下达到”。

从技术选型方面看，当前整体使用 FastAPI 作为后端模式，用 Vue 作为前端模式，还使用 SQLite 当作知识库的存储形式，并且通过大语言模型接口来完成语义推理。这就意味着，要达到系统的要求，就得和现有的技术栈保持一致，不能随便引入和整体有冲突的复杂架构。

从数据和结果格式方面看，整体需要保证生成的结果是结构化的数据。特别是用户故事生成的结果，一定要包含结构化需求、用户故事列表和任务清单等重要字段。任务类型要限制在前端、后端和数据库这三类里面，依赖关系要用标准字段来表示。

从运转环境方面看，整体需要同时考虑本地演示环境和轻量化部署场景，所以数据库应该优先使用轻量级方案，前后端部署流程要尽量简单，防止依赖高成本的基础设施。

从模型调用方面看，因为整体依赖外部大语言模型接口，所以要接受模型响应时间和网络环境对整体响应时间的影响，同时可以在提示词设计和结果规范化处理中增强稳定性。

结合以上约束需求，得到约束需求表如表 3.5 所示。

表 3.5 约束需求表

编号	需求类型	需求内容	说明
CR1	技术约束	后端采用 FastAPI，前端采用 Vue	与现有实现保持一致
CR2	存储约束	知识库采用 SQLite	满足轻量化场景
CR3	输出约束	结果需采用结构化格式返回	便于前端展示与后续处理
CR4	任务约束	任务类型限定为前端、后端、数据库	保持任务拆解统一性
CR5	运行约束	系统依赖外部模型接口	响应时间受网络和模型服务影响

3.3 需求规约

在完成需求分析之后，可以把前面说的需求转化成结构化、能追踪、能实现的需求规约。需求规约的目的是能用明确的语言来描述每一项需求，并且在后续的设计、实现和测试阶段，可以对这些需求进行追踪和验证。

这个整体使用‘编号+需求类型+需求描述+优先级+验收标准’的方式对核心需求进行规约。这里面，业务需求主要是对系统建设目标进行描述，用户需求主要是对用户希望得到的能力进行描述，功能需求主要是对系统应该提供的具体功能进行描述，非功能需求主要是对质量属性要求进行描述，约束需求则是对必须遵守的技术和环境边界进行描述。

在规约的过程中，可以尽量防止使用模糊的表达，像“系统应尽量快”“结果要比较好”这类，而是可以改用能够检查、能够观察的描述方式。比如说，对于生成用户故事的需求，可以表述成“系统应该输出符合标准模板的用户故事文本”。对于拆解任务的需求，可以表述成“系统应该返回包含任务编号、任务类型和依赖字段的任务列表”。对于部署的需求，可以表述成“系统应该支持本地开发启动和容器化部署”。需求规约如表 2.6 所示。

表 3.6 需求规约表

编号	需求类型	需求规约内容	优先级	验收标准
BR1	业务需求	系统应实现从原始需求到用户故事、任务拆解和代码辅助输出的完整闭环	高	能完整完成输入、生成、评估和导出流程
UR1	用户需求	用户应能够输入自然语言需求并获得结构化结果	高	前端支持输入并成功返回生成结果
UR2	用户需求	用户应能够通过代码片段反向生成用户故事	中	支持代码输入并返回故事与任务
FR1	功能需求	系统应生成知识增强上下文	高	结果中包含知识增强信息
FR2	功能需求	系统应生成结构化需求、用户故事和任务清单	高	返回结构中包含对应字段
FR3	功能需求	系统应支持任务依赖关系展示	高	前端能展示依赖图
FR4	功能需求	系统应支持结果评估	高	返回得分、结论和建议
FR5	功能需求	系统应支持用户故事相关代码生成	中	返回项目结构、代码和SQL
NFR1	非功能需求	系统界面应具备良好的可读性和可操作性	中	页面结构清晰，核心区域可区分
NFR2	非功能需求	系统应具备异常处理和统一响应能力	高	异常输入时页面不崩溃，返回错误提示
CR1	约束需求	系统应基于既定技术栈和轻量化部署方案实现	高	可在本地或容器环境中完成部署

3.4 需求验证

需求验证的目的是进行确认，看看所获取和分析的需求是不是正确、完整、一致、可以实现并且可以验证的。需要强调一下，本章进行的需求验证和第 6 章中的系统测试是不一样的。本章里的需求验证主要是验证“需求本身能不能成立、是不是合理、可不可以落地”，而第 6 章会进一步验证“系统实现之后能不能满足这些需求”。

本整体的需求验证主要使用需求审查、用例检查、原型对照、技术可行性分析和需求可追踪性检查这五种方式来进行。需求审查主要是用来检查需求表述是不是清晰，有没有歧义或者重复的情况。用例检查主要是验证用户需求有没有覆盖系统主要交互行为。原型对照主要是验证功能需求能不能在当前页面结构中实现。技术可行性分析是用来验证技术选型和约束需求是否匹配。需求可追踪性检查则是确认每一类需求在后续设计、实现和测试阶段能不能找到对应的支撑。

从验证结果可以看出，这个整体的核心需求之间没有明显的冲突。业务需求给用户需求和功能需求提供了总体的目标。用户需求可以被现有的前端交互流程覆盖，功能需求能够通过现有的接口和模块来达到。非功能需求和约束需求也和当前的技术模式相匹配。所以，本章形成的需求模式有完整性和可行性。后续的章节会分别从系统设计、系统实现和系统测试这三个方面，对这些需求进行进一步的落实和检验。需求验证方式与验收依据如表 3.7 所示。

表 3.7 需求验证与验收表

需求编号	需求内容	设计对应	实现对应	测试对应
FR-01	支持自然语言需求输入与校验	输入模块设计	前端输入区、生成接口	输入有效性测试
FR-02	支持代码片段反推需求	代码输入流程设计	代码分析与故事生成接口	代码输入测试
FR-03	支持知识增强上下文生成与展示	知识增强模块设计	知识匹配逻辑、上下文展示区	知识命中测试
FR-04	支持用户故事生成	生成模块设计	用户故事生成接口	功能测试
FR-05	支持任务拆解与依赖展示	任务与依赖设计	任务清单、依赖图展示	功能测试
FR-06	支持结果评估	评估模块设计	自动化评分与建议输出	结果评估测试
FR-07	支持结果导出	导出模块设计	Markdown 导出功能	导出测试
FR-08	支持知识模块管理	知识库管理设计	知识模块查询与新增接口	接口测试

为了保证在后续的整体设计、达到和测试阶段，需求可以得到有效验证，本文建立了需求追踪矩阵，把需求分析阶段提出来的核心需求，和设计模块、达到的功能以及测试内容做了映射。需求追踪矩阵的作用是保证每一项需求都能在后续章节里找到对应的支撑，防止需求孤立存在，或者达到的内容脱离需求基础。

从追踪关系来说，前端输入模块和后端生成接口可以追踪到自然语言需求输入和校验需求。代码输入模块和代码反推生成接口能够追踪到代码片段反推需求。知识匹配模块和上下文展示模块可追踪到知识增强展示需求。生成模块、任务列表模块和依赖图模块能追踪到用户故事生成、任务拆解和依赖展示需求。自动化评估模块可追踪到结果评估需求。导出模块能追踪到结果导出需求。知识库接口和知识存储模块则可追踪到知识模块管理需求。通过这样的方式，系统需求能从‘提出’进一步落实到‘可设计、可实现、可测试’的层面。

3.5 本章小结

这章围绕整体建设目标，对智能用户故事生成整体做了比较系统的需求分析。首先，从本文目标、软件工程实践的痛点、用户角色场景和技术实现条件来做，完成了需求获取工作。之后，从业务需求、用户需求、功能需求、非功能需求和约束需求这五个方面对整体需求进行分类分析。在这个基础上，通过需求规约把核心需求变成结构化、能追踪的条目。最后，通过需求验证确定了各类需求之间的一致性、合理性和可实现性。

从本章的分析能够知道，这个整体不光要拥有用户故事生成这个重要功能，还得同时满足知识增强、任务拆解、依赖展示、质量评估和代码辅助输出等多方面的要求。这些需求给后面的整体设计、整体实现和整体测试提供了清晰的依据。

第 4 章 系统设计

4.1 概要设计

4.1.1 设计目标

本系统整体的设计目标主要是围绕需求理解的准确性和生成结果的可用性来进行展开的。以自然语言需求和代码片段这两类输入作为起始点，通过引入领域知识库和大语言模型协同推理的方式，达到需求结构化提取、用户故事生成、任务拆解、结果评估和代码辅助生成。使用前后端分离的模式，前端负责交互展示、结果筛选和可视化呈现，后端负责知识增强、模型调用、评估计算，这样可以保证整体有较好的模块解耦性和扩展性。同时，整体引入统一响应结构、知识库初始化、容器化部署和代理转发方式，可以支持本地演示和后续部署应用设计目标和实现策略如表 4.1 所示。

表 4.1 系统设计目标与实现策略表

设计目标	具体说明	实现策略
多源输入支持	支持自然语言需求和代码片段输入	前端设置双输入区域，后端分别提供自然语言生成接口和代码反推接口
知识增强生成	在生成前补充业务规则、异常场景和安全约束	通过数据库知识库匹配模块名与别名，并拼接增强上下文
结构化输出	生成可计算、可展示的标准化结果	统一输出结构化需求、用户故事、任务清单和依赖字段
可视化与评估	提升生成结果的可读性与可分析性	通过故事卡片、任务表格、依赖图和评分模块展示结果
工程辅助能力	让生成结果能够进一步服务开发实现	基于选中用户故事继续生成前端代码、后端代码和 SQL 骨架

4.1.2 系统整体架构

为了保证智能用户故事生成整体的高内聚、低耦合，还有易扩展和易维护性，本整体在架构设计时使用了一种清晰的三层功能模式，总共拆分成前端交互层、核心业务层和数据层。系统总体功能模式图如图 4.1 所示。

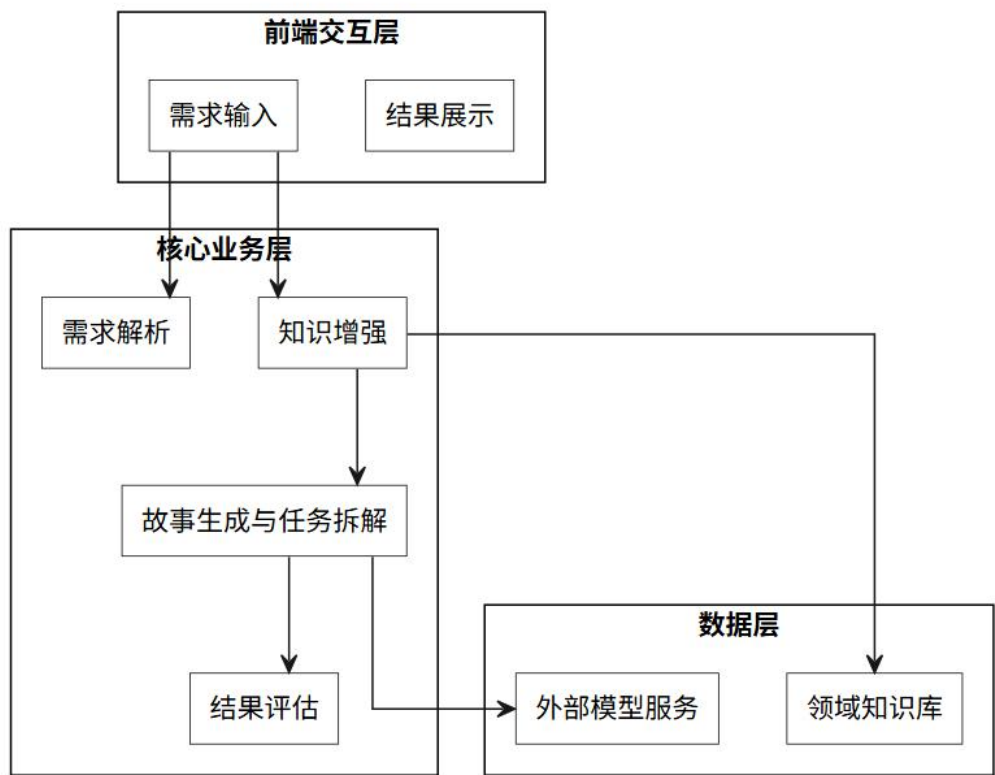


图 4.1 系统总体功能架构图

系统三大核心分层的职责边界与设计规范定义如下：

前端交互层是用户操作和结果呈现的直接形式，它能提供直观的图形界面。它的重要设计任务是提供集中式的自然语言需求输入框，并且负责调用后端的异步接口。在收到返回数据之后，前端交互层可以通过响应式组件，动态地渲染出标准的用户故事卡片、结构化清单表格，还有基于 ECharts 渲染的任务依赖关系拓扑图谱，这样方便用户进行交互式的检查、编辑和导出。

核心业务层是整体的中枢控制大脑，它的作用是把整个智能化需求工程的处理流水线串联起来。核心业务层会对输入的自然语言需求做预处理，把重要要素提炼出来，还会引入数据层的领域知识，进行术语消歧和概念关联。接着，这一层会使用高度工程化的提示词，驱动大模型进行智能推理，自动生成规范的用户故事。然后，再把用户故事进一步细化，拆分成前端、后端、数据库这三类微观任务，形成安全合规的有向无环图（DAG）任务拓扑链。

数据层是整体的资产和规则基础，能给核心业务层提供数据持久化和知识检索方面的支持。数据层的重要部分有轻量级领域知识库和大语言模型 API 这两个

重要供给端。其中，知识库使用 SQLite 和 SQLAlchemy ORM 模式进行实际部署，专门用来存储登录、注册等频繁使用的软件模块的术语、前置条件、异常情况和
安全限制。系统功能规划如表 4.2 所示。

表 4.2 系统功能规划表

模块名称	主要职责	输入	输出
需求文本处理模块	接收自然语言需求或代码片段并进行校验	需求文本、代码文本	合法请求数据
知识增强模块	匹配知识模块并生成上下文	需求文本	增强上下文、命中模块
用户故事生成模块	调用大模型完成结构化生成	需求文本、增强上下文	结构化需求、用户故事、任务清单
结果评估模块	对生成结果进行评分与建议生成	结构化结果	综合分数、维度评分、改进建议
可视化展示模块	以列表、表格和图形方式展示结果	用户故事、任务、评分信息	页面化展示结果
代码生成模块	按选中故事生成相关代码骨架	用户故事、相关任务、原始需求	项目结构、前端代码、后端代码、SQL
知识维护模块	支持知识库查询与新增	模块信息	模块列表、新增结果

4.1.3 技术架构设计

在技术实现上，系统前端采用 Vue 2 构建单页面交互界面，并结合 Element UI 完成输入组件、卡片布局、表格展示和标签化信息表达，结合 ECharts 实现任务依赖关系图的可视化渲染。后端采用 FastAPI 框架提供 REST 风格接口，利用 Pydantic 完成请求数据校验，利用 SQLAlchemy 负责数据库映射与会话管理，借助 OpenAI SDK 调用 DeepSeek 大语言模型接口，实现统一、清晰且便于维护的服务端逻辑。

在部署方式上，系统支持本地开发模式和容器化部署模式。开发环境下前端通过代理将/api 请求转发至本地 FastAPI 服务；部署环境下通过 Docker Compose 编排前后端容器，由 Nginx 提供前端静态资源服务并反向代理后端接口，从而简化系统部署流程并提升演示稳定性。系统技术栈如表 4.3 所示。

表 4.3 系统技术栈表

层次	技术选型	主要用途	选型原因
前端框架	Vue 2	构建单页面交互界面	组件化开发方便，学习与实现成本低
UI 组件库	Element UI	实现表单、卡片、表格、标签等界面元素	组件丰富，适合管理类系统页面
图形可视化	ECharts	绘制任务依赖关系图	图形表达能力强，适合关系结构展示
后端框架	FastAPI	提供 REST 接口服务	开发效率高，支持类型标注与自动校验
数据校验	Pydantic	请求参数建模与校验	可减少非法请求带来的异常
ORM 与数据库	SQLAlchemy+SQLite	知识库持久化存储	实现简单，适合毕设和本地演示
模型调用	OpenAI SDK +DeepSeek	用户故事生成、代码反推、代码骨架生成	接口统一，便于接入对话模型
部署编排	Docker Compose	前后端容器化编排	便于一键启动和答辩演示
反向代理	Nginx	静态资源服务与 API 转发	方便实现前后端同域访问

4.1.4 系统业务流程设计

系统业务流程可分为自然语言需求生成流程和代码片段反推生成流程两条主线。对于自然语言输入场景，系统首先接收用户输入的需求文本，并调用知识增强模块生成上下文提示信息，随后将原始需求与增强上下文共同输入大语言模型，生成结构化需求、用户故事和任务清单，再由评估模块对结果进行量化评分并返回前端展示。对于代码片段输入场景，系统首先利用模型将代码抽取为简洁的自然语言需求，再进入与自然语言输入相同的知识增强与生成链路。当前端接收到结果后，用户可继续选择某一条用户故事触发相关代码骨架生成，也可导出 Markdown 结果。

4.2 详细设计

4.2.1 前端设计

整体前端使用顶层组件集中管理的方式，来进行页面状态组织和接口协同。顶层页面要负责维护一些核心状态，像需求文本、代码输入、知识增强结果、结

构化需求、用户故事、任务清单、评估结果和生成代码等。它会通过统一请求函数来进行接口访问，通过统一结果标准化函数来适配不同的后端响应结构，这样可以防止各展示组件直接和后端返回格式耦合。

用户故事列表组件可以对故事卡片进行展示，还能同步选中状态。任务清单组件可以对任务进行过滤和搜索，还能反向定位故事。任务图组件可以把任务和它的依赖关系构造成图结构，并且能响应节点点击事件。采用这种组件拆分方式，前端可以保持单页应用的连贯性，也能有较好的职责边界。前端核心组件职责如表 4.4 所示。

表 4.4 核心组件职责表

组件或文件	主要职责	关键功能
APP.vue	前端总控页面	状态管理、接口请求、结果标准化、页面联动、导出 Markdown
StoryList.vue	用户故事列表展示	故事卡片展示、选中同步、角色与目标提取
TaskList.vue	任务清单展示	任务筛选、关键字搜索、故事反向定位
TaskGraph.vue	任务依赖图可视化	构建图节点与边、颜色分类、点击联动
main.js	前端入口初始化	挂载 Vue 实例并注册 Element UI

主页面使用分区式的信息布局方式，顶部是整体的标题和功能说明，中部有自然语言需求输入卡片和代码片段输入卡片，能进行示例填充、清空和生成操作。下部会依次展示知识增强上下文、结构化需求表示、用户故事列表、任务清单、相关代码生成结果、任务依赖图和实验评估结果。页面还提供结果导出和页面重置功能，可以支持实验记录和多轮对比

在界面交互方面，整体使用响应式栅格布局的方式，对桌面端和移动端进行适配。用户选好某条用户故事之后，任务清单、依赖图和代码生成区域会围绕该故事联动起来，提高了结果浏览的效率。整个页面从输入、生成、评估到辅助代码输出，形成了一个连续的业务闭环。页面区域与交互说明如表 4.5 所示。

表 4.5 页面交互说明表

页面区域	主要功能	交互说明
自然语言需求输入区	输入和提交需求文本	支持示例填充、清空和生成
代码片段输入区	输入已有代码并反推需求	支持示例填充、清空和生成
知识增强上下文区	显示命中的知识规则	直观展示必选要素、前置条件和异常场景
结构化需求区	展示角色、行为、条件和目标	通过标签形式呈现结构化结果
用户故事列表区	展示全部用户故事	点击卡片可选中当前故事
任务清单区	表格化展示任务信息	支持按类型筛选和关键字搜索
代码生成区	显示选中故事对应代码骨架	以标签页形式展示项目结构、前端代码、后端代码和 SQL
任务依赖图区	显示故事与任务间依赖关系	点击节点可反向选中故事
实验评估区	展示得分、结论和建议	用于辅助分析生成质量
全局操作区	导出与重置	支持结果导出为 Markdown 文本

4.2.2 后端设计

后端逻辑可以划分成三个部分，分别是接口层、业务处理层和数据持久层。接口层主要是由主应用文件里的各种路由构成的，它负责接收请求、校验参数、捕获异常，还有封装统一响应。业务处理层主要包含知识增强模块、大语言模型生成模块等。数据持久层是由数据库连接配置、知识库模型定义和知识初始化脚本组成的。

这种分层的方式可以让不同职责的代码进行相对独立的演化。比如说，知识库匹配逻辑发生变更，不会直接对接口层产生影响。对模型输出结构进行规整化处理，也集中在业务处理模块里，这样就减少了前端和模型返回结果之间的耦合程度。后端接口设计如表 4.6 所示。

表 4.6 后端接口设计表

接口路径	请求方法	功能说明	核心请求字段	主要响应内容
/health	GET	健康检查	无	服务状态
/knowledge_modules	GET	查询知识模块	无	模块总数、模块明细
/knowledge_modules	POST	新增知识模块	module_name、category、aliases	新增模块信息

表 4.6 后端接口设计表（续）

接口路径	请求方法	功能说明	核心请求字段	主要响应内容
/preview_context	POST	预览知识增强上下文	requirement	知识增强上下文
/generate_story	POST	生成用户故事与任务拆解	requirement	知识上下文、结构化结果
/generate_batch	POST	批量生成多条需求结果	requirements	每条需求的生成结果与成功状态
/generate_and_evaluate	POST	生成结果并进行评估	requirement	结构化结果、评估结果
/generate_story_from_code	POST	从代码片生成用户故事	code	抽取后的需求、结构化结果、评估结果
/generate_code	POST	基于选中用户故事生成代码骨架	requirement、story、tasks	项目结构、前后端代码、SQL、使用说明

后端分层调用关系图如图 4.2 所示。

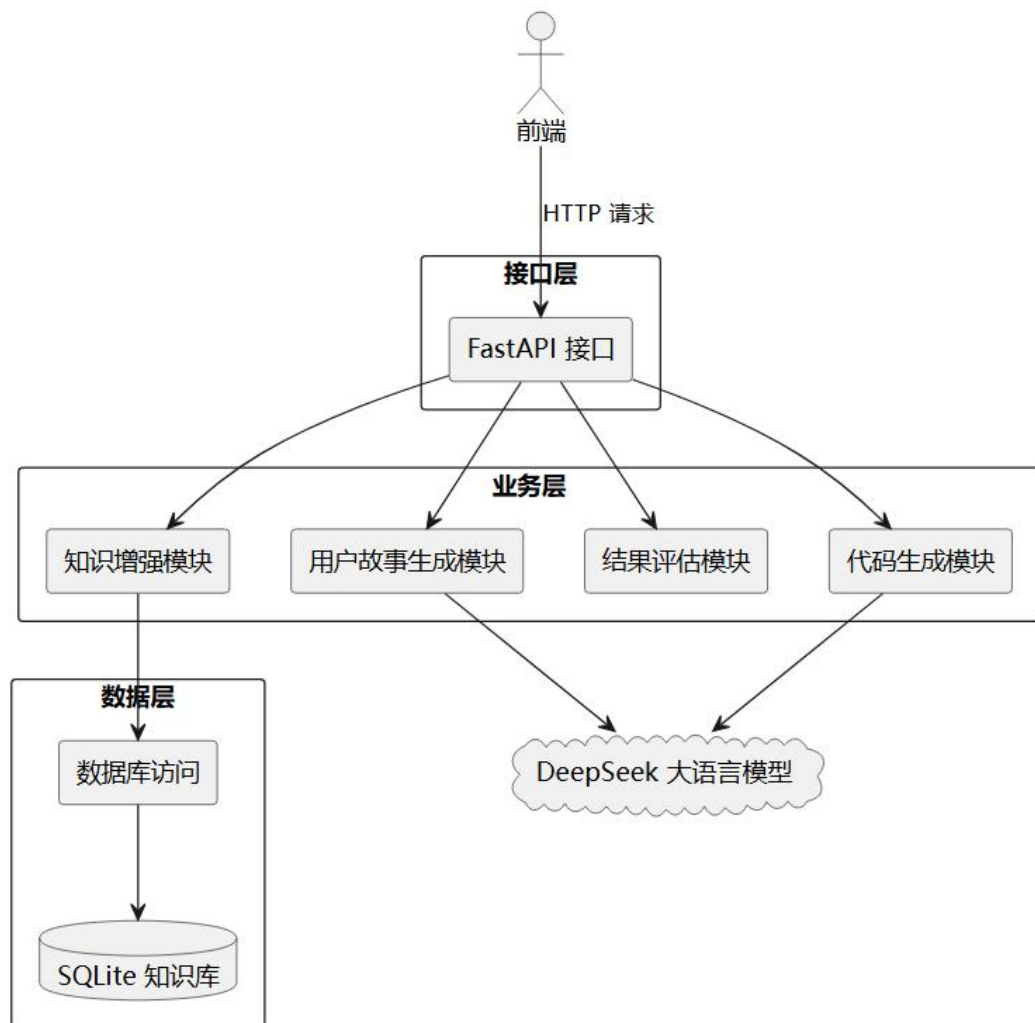


图 4.2 后端分层调用关系图

4.2.4 知识增强模块设计

知识增强模块是这个整体和直接调用大语言模型生成的不同之处，是重要的设计。系统启动的时候，会自动对知识库数据进行初始化，把登录、注册、支付、审批流、报表统计这些常见的软件业务场景写到数据库里。要是命中了某个业务模块，系统就会从数据库里提取这个模块对应的必选要素、前置条件、异常场景、典型任务和安全约束，然后把它们拼起来，形成增强上下文，给后面的模型推理使用。为了让“知识融入”不只是在概念表述层面，系统还对知识库规模和规则构成做了量化统计。内置的知识库一共有 38 个知识模块，可以覆盖 17 类典型业务场景。这里面累计有 204 个模块别名、186 条必选要素、73 条前置条件、152 条异常场景和 156 条典型任务。规则类型有必选要素、前置条件、异常场景、典型任务和安全约束这 5 类。上面这些量化信息会通过系统接口统一输出，可以作为判断知识库初始化是否成功、知识覆盖范围是否足够，还有后续实验中分析“知识融入”贡献度的直接依据。

要是需求在知识库中没找到对应的内容，系统也会给出通用的软件工程提示，这样可以保证流程能够继续执行。这个模块有效地减少了模型遗漏重要业务规则的可能性，提高了用户故事和任务拆解的完整性。知识库数据表字段设计如表 4.7 所示。

表 4.7 数据库字段设计表

字段名	数据类型	含义说明
id	Integer	主键编号
module_name	String(100)	知识模块名称
category	String(100)	知识模块所属类别
aliases	Text	模块别名列表，采用 JSON 字符串存储
required_elements	Text	必选要素列表，采用 JSON 字符串存储
preconditions	Text	前置条件列表，采用 JSON 字符串存储
exception_scenarios	Text	异常场景列表，采用 JSON 字符串存储
typical_tasks	Text	典型任务列表，采用 JSON 字符串存储
security_constraints	Text	安全约束说明
description	Text	模块描述信息
is_builtin	Boolean	是否为系统内置知识模块
created_at	DateTime	创建时间
updated_at	DateTime	更新时间

4.2.3 需求文本处理模块设计

在智能用户故事生成整体中，自然语言需求的预处理是整个流程里的基础步骤，它的质量会直接影响之后的结构化表示，还有任务推理的正确性。原始需求一般是从产品经理文档、用户访谈或者会议纪要等地方来的，存在非结构化、语义松散、表达重复或者语法错误等问题。所以，系统会先对输入的文本进行分句操作，把一个长段落分成一个个独立的句子，这样方便之后逐句分析语义。需求文本处理模块的作用是，保证输入数据在进入模型之前，有统一、稳定且能处理的结构。对于自然语言输入，系统会先对文本进行去空格和非空校验，防止无效请求进入模型流程。对于代码片段输入，系统会调用需求抽取逻辑，把代码内容转换成 1 到 3 句中文业务需求描述，然后再送到知识增强与生成链路中。在结果处理阶段，系统会对模型输出的结构化字段、用户故事列表和任务列表进行标准化整理，特别是对任务编号、任务类型和依赖字段进行安全转换和合法性修正，保证后续的表格展示、图形形成和评估计算能够稳定进行。需求文本处理流程如图 4.3 所示。

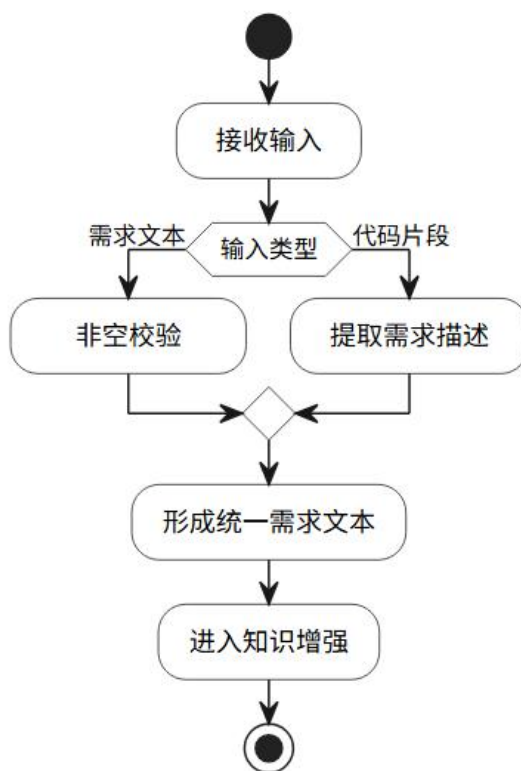


图 4.3 需求文本处理流程图

整体做好准备 38 个知识模块已经准备好，这些模块覆盖了 17 类典型场景，像安全认证、电商交易、后台管理、流程协同、教育系统、政务系统等。开展设计工作设计了内置知识模块分类统计如表 4.8 所示。

表 4.8 模块分类统计表

指标名称	数量	指标数值说明
知识模块总数	38	包含登录、注册、支付、审批流、报表统计等常见业务必选要素、前置条件、异常场景、典型任务、安全约束
业务分类数	17	覆盖安全认证、电商交易、后台管理、流程协同、教育系统、政务系统等模块
模块别名总数	204	用于归一化匹配和近义表达命中
必选要素总数	186	用于补全功能点与关键业务字段
异常场景总数	152	用于提升失败处理与边界场景
前置条件总数	73	用于补充使用前提和状态约束
典型任务总数	156	用于增强任务拆解质量
规则类型数	5	必选要素、前置条件、异常场景、典型任务、安全约束

输入需求文本后调用知识库，知识增强流程如图 4.4 所示。

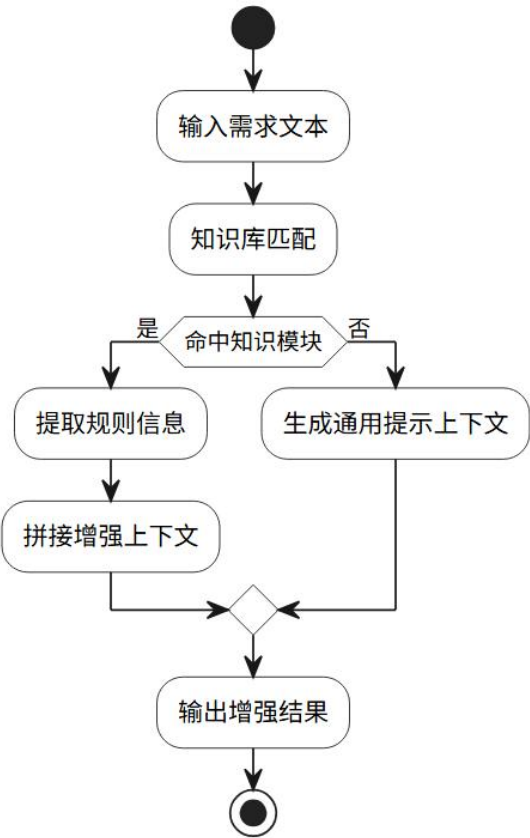


图 4.4 知识增强流程图

4.2.5 大语言模型生成模块设计

大语言模型生成模块会把原始需求和知识增强上下文结合起来，转化成结构化的输出。系统会在提示词里明确要求模型必须用中文，只能返回合法的 JSON 对象，用户故事要按照固定模板来，任务类型只能选前端、后端和数据库这三类，还要求依赖字段明确表示出前置任务编号。

在模型调用完成之后，系统会进一步对返回的结果进行 JSON 解析，并且进行规范化处理。要是检测到 API Key 没有配置，或者模型返回的内容不是合法的 JSON，又或者生成的结果为空，系统就会返回带有错误标记的默认结构，这样能保证前端可以进行统一处理。在目前的实现当中，生成模块除了返回规范化的结果之外，还会记录模型调用的耗时、提示词的长度、是否实际调用了模型、故事的数量、任务的数量以及错误的类型等跟踪信息。这样做可以在系统实现阶段直观地展示生成链路的运转证据，也可以在第 6 章性能测试中把“模型推理耗时”和“本地处理耗时”区分开来，防止只使用总响应时间掩盖系统真正的瓶颈。这种“强约束生成+异常兜底”的设计提高了系统的鲁棒性。大模型生成与规范化流程如图 4.5 所示。

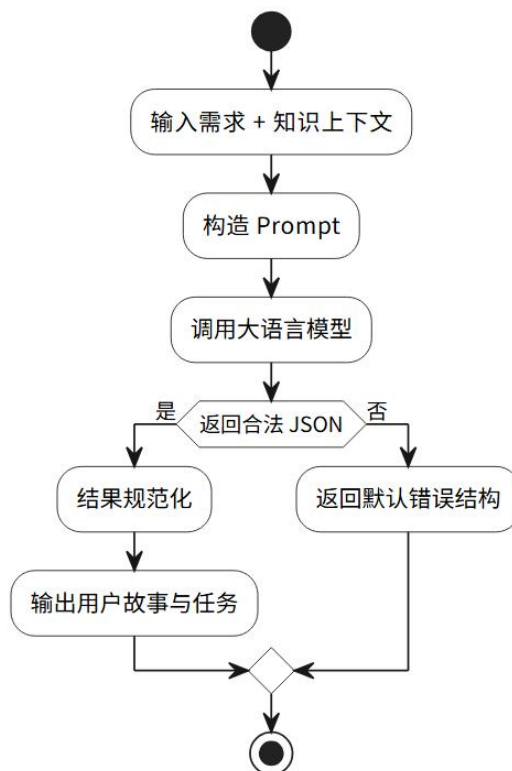


图 4.5 大语言模型生成与规范化流程图

4.2.6 用户故事生成与结果评估设计

在用户故事生成阶段，系统将输出统一约束为“作为某类角色，我想要完成某项功能，以便获得某种业务价值”的规范表达形式。与用户故事同步生成的任务清单要求包含任务编号、任务名称、任务类型、所属故事和依赖关系，便于后续进行分工、排序与可视化分析。当前实现中，任务类型被限制为前端、后端和数据库三类，既符合软件开发任务的常见划分方式，也方便前端界面进行颜色区分与筛选展示。用户故事与任务依赖可视化设计如图 4.6 所示。

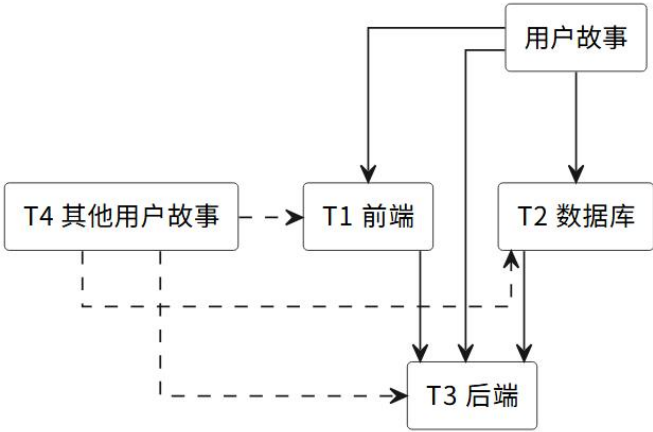


图 4.6 用户故事与任务依赖可视化图

生成完成后，系统进一步提供结果评估和辅助代码生成能力。评估模块从结构化完整度、用户故事质量、任务拆解质量和依赖关系质量四个维度对输出结果进行打分，并给出针对性的改进建议；当用户选择某条用户故事后，系统还可进一步生成与该故事相关的前端代码、后端代码和数据库 SQL 骨架。结合任务依赖图的图形化展示，系统实现了从需求输入到结果可视化和工程辅助输出的完整闭环。评估维度与评分规则如表 4.9 所示。

表 4.9 评估维度与评分规则表

评估维度	分值	评分规则
结构化完整度	35	角色 10 分，行为 10 分，条件 5 分，目标 10 分
用户故事质量	25	判断是否生成故事，以及是否满足“作为、我想要、以便”的模板要求

表 4.9 评估维度与评分规则表（续）

评估维度	分值	评分规则
任务拆解质量	20	任务数量达到 2 条及以上，且任务类型合法
依赖关系质量	20	任务是否包含 <code>depends_on</code> 字段且格式正确
综合建议生成	不单独计分	根据缺失信息输出改进建议
总计	100	当前实现采用四维加权累加方式进行评分

4.2.7 代码辅助生成模块设计

在智能用户故事生成整体里，用户故事和任务拆解的结果已经可以给需求分析和任务规划提供支持。不过，如果系统的输出能进一步为实现阶段服务，它的工程应用价值会得到进一步提高。基于这个考虑，本文在系统设计时增加了代码辅助生成模块。这个模块的作用是，在用户选定某条用户故事后，结合其对应的任务信息，进一步生成相关的前端代码、后端代码、数据库 SQL 以及项目结构建议，这样就能从需求表达延伸到实现参考。

在触发方式方面，代码辅助生成模块不是独立运转的，而是依赖于用户故事生成结果。用户完成需求输入、知识增强、用户故事生成和任务拆解后，前端页面会让用户从用户故事列表、任务清单或者依赖关系图里选中某一条目标用户故事，然后把这个故事及其关联任务当作输入，提交给后端代码生成接口。后端收到请求后，会把原始需求、目标故事文本和任务清单一起组织成模型提示信息，要求模型从实现角度输出对应的代码骨架和结构建议。通过这样的设计，代码辅助生成模块不会脱离原有的需求分析链路，而是作为结果深化和工程参考的一部分，嵌入到整体业务闭环中。

在输出内容设计方面，代码辅助生成模块主要有四类结果。一种是项目结构建议，它能说明当前用户故事可能会涉及的目录组织和模块划分。二，前端代码，可以展示页面交互、表单输入或者展示逻辑的实现参考。第三，后端代码，能提供接口定义、业务处理逻辑或者服务层骨架。还有数据库 SQL，可给出表结构设计或者核心数据字段定义。必要的时候，系统还可以添加使用说明或者后续扩展

提示，来帮助用户理解生成代码的适用范围和使用方法。该模块的目标不是直接替代人工开发，而是给用户提供和当前故事紧密相关的实现草案和开发参考，这样可以提高从需求到实现的衔接效率。

从整体设计方面看，代码辅助生成模块让整个系统“需求分析—故事生成—任务拆解—工程辅助输出”的完整链路得到了进一步强化。它让系统输出结果不只是停留在文本层面，还具备了向开发实现延伸的能力。同时，也提高了系统在教学演示、原型验证和敏捷开发前期分析中的综合应用价值。所以，代码辅助生成模块是这个系统和一般用户故事生成工具不同的重要扩展设计之一。

4.3 本章小结

本章围绕整体设计和重要模块设计展开说明，给出了整体的模式、技术模式、业务流程和前后端详细的设计方案通过前后端分离模式、知识增强方式、结构化生成约束和可视化展示设计，整体从自然语言需求或代码片段开始，自动生成用户故事、任务拆解结果并进行质量评估，还能从用户故事反向生成代码，完成了完整的流程这为后续系统实现、系统测试和结果分析打下了基础。

第 5 章 系统实现

5.1 系统整体架构实现

整体达到阶段主要是围绕数据层、核心业务层和前端交互层这三个部分来进行展开的。在具体达到方面，数据层使用 SQLite 和 SQLAlchemy 来完成知识库数据的持久化管理，核心业务层使用 FastAPI 形成接口服务，负责请求接收、知识增强、模型调用、结果评估和代码生成。前端交互层使用 Vue2 和 Element UI 形成单页面交互界面，并且结合 ECharts 达到任务依赖关系的图形化展示。这三个部分协同之后，整体可以完成从需求输入到用户故事生成、任务拆解、质量评估和结果展示的完整处理流程。

整体没有只是进行原型级页面拼接，而是通过模块拆分、统一响应封装、数据库初始化、代理配置和容器化部署等方式，形成了相对完整的工程化达到基础，给后续功能扩展和部署迁移提供了比较好的支撑。

5.1.1 数据层的初始化

整体数据层达到目的主要依靠 SQLite 数据库和 SQLAlchemy 对象关系映射方式。在 database.py 中，整体通过 create_engine、SessionLocal 和 Base 进行数据库连接、会话管理和模型基类定义。其中，数据库默认使用 SQLite，这样既减少了部署复杂度，也满足了这个整体在知识存储和本地演示方面的轻量化需求。

在 knowledge_models.py 里，整体定义了 KnowledgeModule 实体类，并且把它映射到 knowledge_module 表。这个表主要用来保存通用软件业务模块的领域知识，这里面有模块名称、类别这些内容，还有必选要素、前置条件等。考虑到 SQLite 对复杂数组类型的支持是有限的，整体把 aliases、required_elements 等字段都统一序列化成本文格式来存储，这样既保证了结构表达能力，又保证了和数据库的兼容性。

为了保证整体在第一次运转的时候有可以使用的先验知识，整体在 main.py 的启动逻辑里自动进行建表和知识初始化的操作，还通过

`seed_common_knowledge.py` 把登录、注册、订单管理、支付、审批流、工单管理等 38 个内置知识模块写到数据库里。这种设计防止了整体运转后知识库是空的问题，给后面的知识增强和用户故事生成提供了稳定的数据基础 `main.py` 在整体部署或者演示之前，通过执行 `seed_common_knowledge.py` 脚本里的 `seed_common_knowledge()` 函数，触发基线数据的导入。这个函数利用持久化会话对安全认证、电商订单支付、用户管理、审批工单等核心模块进行检查。要是数据库里没有对应的模块，就通过 ORM 持久化插入；要是有了，就进行平滑更新，保证了整体运转前的先验知识是完备的 `seed_common_knowledge.py` 成功部署知识库数据库如图 5.1 所示。

```
PS C:\Users\翁艺恺\Desktop\毕设论文\UserStoryLLM\backend> uvicorn main:app
INFO: Started server process [58940]
INFO: Waiting for application startup.
通用知识库初始化完成: 新增 0 条, 更新 38 条, 总计 38 条。
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:52933 - "OPTIONS /generate_and_evaluate HTTP/1.1" 200 OK
INFO: 127.0.0.1:52933 - "POST /generate_and_evaluate HTTP/1.1" 200 OK
INFO: 127.0.0.1:51782 - "POST /generate_and_evaluate HTTP/1.1" 200 OK
INFO: 127.0.0.1:60721 - "POST /generate_and_evaluate HTTP/1.1" 200 OK
INFO: 127.0.0.1:49942 - "OPTIONS /generate_story_from_code HTTP/1.1" 200 OK
```

图 5.1 数据库部署成功图

5.1.2 核心业务层

整体核心业务层主要是由接口调度、知识增强、结构化生成、结果评估和代码生成这五个部分构成的。接口入口集中在 `main.py` 里进行定义，系统通过统一的请求模型和响应结构，来达到对不同业务功能的封装。目前后端已经实现了健康检查、知识库查询与新增、知识增强预览、用户故事生成、批量生成、生成并评估、从代码生成用户故事以及按故事生成相关代码等接口，可以满足系统核心功能的调用需求。

在具体的处理流程方面，当用户把自然语言需求输入后，后端会先进行调用 `kb_module.py` 来对需求文本进行归一化处理，还会依据模块名和别名完成知识匹配，进而生成知识增强上下文，之后由 `llm_module.py` 来构造提示词，并且调用 DeepSeek 模型，然后生成结构化需求、用户故事和任务清单。接着再由 `evaluation_module.py` 对结果做打分，还会生成建议。如果用户选中某条用户故事

并要继续开发，系统会进一步调用 `code_module.py` 输出前端代码、后端代码和数据库 SQL 骨架。后端核心业务调用关系如图 5.2 所示。

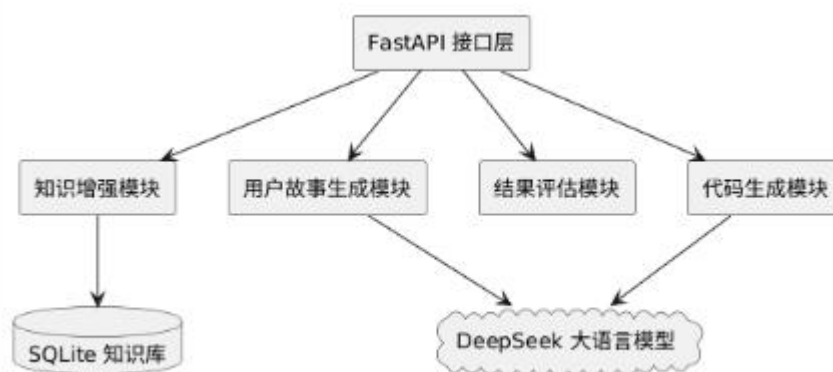


图 5.2 后端核心业务调用关系图

为了提高后端服务的稳定性，系统在模型输出之后，增加了结果规范化和异常兜底处理的方式。要是模型返回的内容不符合 JSON 格式，或者 API Key 没有配置，系统会返回默认结构和错误信息，不会造成前端渲染失败。这种方式明显提高了系统在演示场景中的可用性和鲁棒性。后端核心服务模块说明如表 5.1 所示。

表 5.1 后端核心服务模块说明表

模块文件	核心函数或对象	主要职责	与其他模块的关系
main.py	路由函数、请求模型、统一响应函数	接收请求、封装返回结果	连接所有业务模块
kb_module.py	get_enhanced_context、extract_matches	进行知识匹配与上下文拼接	为生成模块提供增强信息
llm_module.py	build_prompt、generate_structured_data、extract_requirement_from_code	调用大模型生成结构化结果	接收知识上下文并输出故事与任务
evaluation_module.py	evaluate_result	对生成结果进行量化评分	处理生成模块的输出
code_module.py	generate_related_code	针对选中用户故事生成代码骨架	接收故事与任务列表继续生成
database.py	engine、SessionLocal、Base	提供数据库连接与会话能力	为知识库模块和初始化模块服务

5.1.3 前端交互层的界面呈现

前端交互层主要是由顶层页面和三个子组件构成的在 APP.vue 里，系统会对需求文本、代码输入、知识增强上下文、结构化需求结果、用户故事列表、任务清单、评估信息和生成代码等核心状态进行统一管理，并且通过统一请求函数来完成对后端接口的访问为了让不同接口的返回结构能够适配，前端还进行了结果标准化与统一应用方式的实现，这样可以保证在不同功能场景下，页面展示逻辑能够保持一致。

在组件划分这方面 StoryList.vue 负责展示用户故事卡片，还能选中当前故事 TaskList.vue 负责以表格形式展示任务，并且支持类型筛选和关键字检索 TaskGraph.vue 负责依据 ECharts 绘制故事节点、任务节点以及依赖关系边，还能通过节点点击事件带动页面联动。这种‘顶层集中管理状态，子组件负责展示和局部交互’的方式，让前端页面在进行功能扩展时，有更好的可维护性前端组件交互关系如图 5.3 所示。

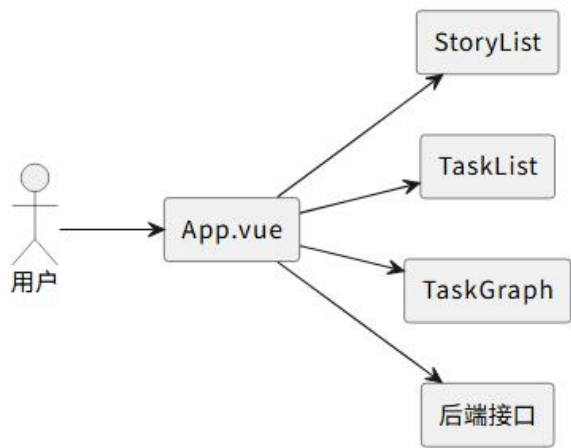


图 5.3 前端组件交互关系图

另外，前端还达到了示例填充、页面重置、结果导出和代码生成这些功能。用户可以通过自然语言需求入口来完成用户故事生成，也能通过代码片段入口反向抽取业务需求，然后继续进行结果查看和代码骨架生成，形成一个完整的前端交互闭环。

5.2 各功能实现模块

本系统是围绕智能用户故事生成这个核心目标来构建的，落地了自然语言需求处理、代码片段反向分析、知识增强、结果可视化展示、质量评估以及代码辅助生成等多个功能模块。为了避免在功能实现的部分，仅仅停留在概念描述的层面，论文对每项核心功能都采用了“输入样本、运行截图、关键返回结果、对应的实现模块”这四类证据放在一起进行说明的方式。其中，自然语言需求生成和代码片段反向分析这两个模块，主要关联的是登录与个人信息业务场景，而知识增强、异常场景补充、任务依赖、结果评估和代码辅助生成这些模块，则统一放到了订单支付这个场景下进行验证，这样做就保证了功能截图、返回的数据和实验样本之间，能形成一条一一对得上的证据链条。各个模块并不是互相孤立的单元，而是沿着“输入处理—知识补充—模型生成—结果展示—质量反馈—工程辅助”这样一条链路在协同工作，从而已经形成了一个比较完整的功能闭环。跟那些只提供单一文本生成能力的系统一比，本系统更加看重的是生成结果的结构化表达、可解释性以及给后续开发工作带来的可用性。

5.2.1 自然语言需求生成功能实现

自然语言需求生成功能是系统的核心入口之一。用户在前端页面输入一段业务需求文本后，系统首先对输入内容进行非空校验，避免无效请求进入后端处理流程。随后，前端将需求文本提交至后端接口，后端根据需求内容调用知识增强模块和用户故事生成模块，对输入进行进一步处理。

在生成阶段，系统将原始需求和知识增强上下文共同送入大语言模型，要求模型输出结构化需求表示、用户故事列表和任务拆解结果。其中，结构化需求主要包含角色、行为、条件和目标四类信息；用户故事采用统一模板表达；任务清单则进一步体现前端、后端和数据库三个层面的开发任务划分。前端生成功能实现如图 5.4 所示。



图 5.4 自然语言需求生成成功图

部分核心代码如表 5.2 所示。

表 5.2 自然语言需求生成核心代码表

```
@app.post("/generate_and_evaluate")
def generate_and_evaluate(req: RequirementRequest):
    requirement = req.requirement.strip()
    if not requirement:
        return error_response("需求文本不能为空")
    total_start = time.perf_counter()
    payload = get_enhanced_payload(requirement)
    generation = generate_structured_data_with_trace(
        requirement,
        payload["context"],
        strategy="knowledge_enhanced"
    )
    result = generation["result"]
    evaluation = None
    if not result.get("error"):
        evaluation = evaluate_result(result)
    return success_response(
        {
            "requirement": requirement,
            "knowledge_context": payload["context"],
            "result": result,
            "evaluation": evaluation
        }, """
        "生成与评估完成"
    """
    )
```

5.2.2 代码片段反向生成用户故事功能实现

除自然语言输入外，系统还支持通过代码片段反向生成用户故事。这一功能主要面向已有系统需求补录、旧系统理解以及代码驱动需求分析等场景。用户可以在前端页面输入一段后端接口代码、前端组件代码或数据库相关代码，系统会先对代码内容进行初步校验，再调用模型提炼其对应的业务需求描述。

在具体实现中，系统首先利用大语言模型从代码中抽取 1 至 3 句中文业务需求概述，将代码逻辑转换为更适合需求分析的自然语言描述；随后，系统复用自然语言需求生成链路，对提取后的需求继续执行知识增强、结构化生成和质量评估。这样既避免了重复开发两套完全独立的处理流程，也保证了自然语言输入与代码输入两种场景下结果格式的一致性。代码选择“个人信息接口”如图 5.5 所示。

 **代码片段输入**

```
@app.get("/user/profile")
def get_profile(current_user: User =
Depends(get_current_user)):
    return {
        "username": current_user.username,
        "email": current_user.email,
        "avatar": current_user.avatar
    }
```

快速示例：

登录 API

购物车 API

个人信息接口

从代码生成故事

清空

图 5.5 代码反向生成需求输入图

生成成功如图 5.6 所示。



图 5.6 代码生成用户故事成功图

该功能的实现增强了系统的适用范围，使其不仅能够处理新需求分析任务，还能在一定程度上承担已有软件代码的业务意图反推工作，提高了系统在软件工程教学和实践场景中的应用价值。部分核心代码如表 5.3 所示。

表 5.3 代码反向生成用户故事核心代码表

<pre>@app.post("/generate_story_from_code") def generate_story_from_code(req: CodeRequirementRequest): code = req.code.strip() if not code: return error_response("代码文本不能为空") extracted_req = extract_requirement_from_code(code) if not extracted_req: return error_response("未能从代码中提取到有效需求描述") payload = get_enhanced_payload(extracted_req) generation = generate_structured_data_with_trace(extracted_req, payload["context"], strategy="code_to_story") return success_response({ "extracted_requirement": extracted_req, "knowledge_context": payload["context"], "result": generation["result"] },</pre>

5.2.3 知识增强与结构化结果展示功能实现

为了提升生成结果的业务针对性，系统在生成链路中增加了知识增强机制。知识增强并非简单关键词拼接，而是结合知识库中的领域规则进行补充。当前系统已构建包含多个业务场景的通用知识模块，多个常见领域。

当用户提交需求后，系统会对文本进行归一化处理，并根据模块名称及其别名进行匹配。如果匹配成功，系统将自动整理对应的知识要素，并生成可直接用于提示词构造的上下文内容。如果未匹配到具体模块，系统也会返回通用的软件工程提示信息，以保证后续流程能够继续进行。前端展示如图 5.7 所示。

【模块】登录
【分类】安全认证
【必选要素】账号密码输入、验证码校验、多因素认证、密码加密存储、错误重试限制
【前置条件】用户已注册、账号处于可用状态
【异常场景】密码错误、验证码失效、账号被冻结、连续输入失败超过限制、账号不存在
【典型任务】设计登录页面、实现登录接口、设计用户表字段、实现失败提示与重试限制、接入验证码校验
【安全约束】需进行身份认证、密码加密存储和失败次数限制

图 5.7 知识增强展示图

结构化需求结果则采用标签化方式分别展示角色、行为、条件和目标。结构化展示如图 5.8 所示。



图 5.8 结构化表示图

知识增强匹配模块别名核心代码如表 5.4 所示。

表 5.4 知识增强核心代码表

```

def extract_matches(requirement: str, knowledge_data: Dict[str, Any]) ->
List[Dict[str, Any]]:

    matched_modules = []
    matched_names = set()

    for module_name, rules in knowledge_data.items():
        aliases = rules.get("aliases", [])
        keywords = [module_name] + aliases

        for keyword in keywords:
            if keyword_hit(requirement, keyword):
                if module_name not in matched_names:
                    matched_modules.append({"module": module_name, "rules":
rules})
                    matched_names.add(module_name)
                    break

        return matched_modules

# 未命中

if not matched_modules:
    return {
        "matched_modules": [],
        "context": "当前需求未命中专用知识库，请仅基于通用软件工程知识进行需求
解析和任务拆解。"
    }

    context_blocks = [format_context(item["module"], item["rules"]) for
item in matched_modules]
    matched_names = [item["module"] for item in matched_modules]

    return {
        "matched_modules": matched_names,
        "context": "\n\n".join(context_blocks)
    }

```

5.2.4 用户故事与任务拆解展示功能实现

用户故事展示模块主要负责以直观的方式呈现生成后的用户故事结果。系统将每条用户故事按照卡片形式展示，并从故事文本中自动提取角色信息和目标信息，辅助用户快速浏览结果重点。当用户点击某一条用户故事时，系统会将其设置为当前选中故事，并驱动后续任务清单和代码生成区域进行联动。任务故事卡片展示如图 5.9 所示。



图 5.9 用户故事卡片图

任务拆解展示模块则以表格化方式呈现系统生成的任务列表。每条任务均包含任务编号、任务名称、任务类型、所属故事和前置依赖关系等信息。为了提高结果筛选效率，系统进一步支持按任务类型过滤和按关键字检索，使用户能够快速定位所关心的前端任务、后端任务或数据库任务。任务清单展示如图 5.10 所示。



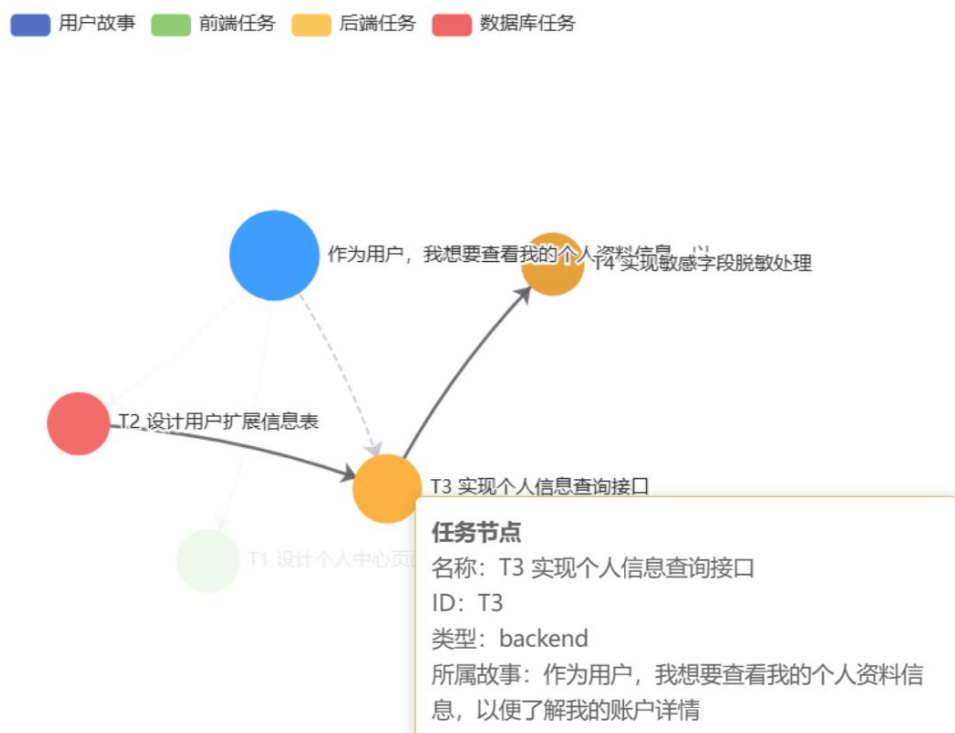
图 5.10 任务清单图

相比仅输出一段完整文本的生成方式，这种“故事卡片+任务表格”的组合展示更加符合软件项目管理和敏捷开发中的信息组织习惯，能够帮助用户从不同层次理解系统输出的开发需求。

5.2.5 任务依赖可视化功能实现

在任务拆解的基础上，系统进一步实现了任务依赖关系可视化展示。该功能基于任务中的依赖字段构建关系图，将用户故事节点作为上层抽象节点，将具体任务作为执行节点，并通过连线表示故事与任务之间的归属关系以及任务之间的前后依赖关系。

在图形实现上，系统使用图可视化组件对不同类型的任务节点进行颜色区分，例如前端任务、后端任务和数据库任务采用不同颜色表示，从而使开发职责划分更加清晰。图中节点支持点击交互，用户点击某个故事节点或任务节点后，系统会自动同步当前选中的用户故事，实现图形视图与列表视图之间的联动。任务依赖可视化如图 5.11 所示。



该功能的实现提升了系统结果的表达层次，使原本线性排列的任务结果转化为可观察、可分析的关系结构，有助于用户理解任务之间的依赖顺序和实现逻辑，也为后续项目分工与开发排期提供了一定参考。任务拆解核心代码如表 5.5 所示。

表 5.5 任务拆解核心代码表

```
const links = []
const storySet = new Set()
const taskNodeMap = new Map()
this.tasks.forEach(task => {
  const taskNodeName = `${task.id || ''} ${task.name || ''}`.trim()
  const storyName = task.story || '未归类故事'
  if (!storySet.has(storyName)) {
    nodes.push({ name: storyName, category: 0, symbolSize: 60 })
    storySet.add(storyName)}
  if (!taskNodeMap.has(task.id)) {nodes.push({
    name: taskNodeName,
    value: task.id,
    category: this.getCategoryIndex(task.type)}})
  taskNodeMap.set(task.id, taskNodeName)
  links.push({ source: storyName, target: taskNodeName })})
return { nodes, links }
```

5.2.6 结果评估功能实现

为了避免系统只输出生成结果而缺乏质量反馈，本系统增加了结果评估功能。评估模块在生成完成后，自动从结构化完整度、用户故事质量、任务拆解质量和依赖关系质量四个维度对结果进行量化分析，并输出综合得分、文字化结论和改进建议。

其中，结构化完整度主要考察角色、行为、条件和目标是否被有效提取（35分）；用户故事质量主要考察故事是否符合规范模板（25分）；任务拆解质量主要关注任务数量和类型划分是否合理（20分）；依赖关系质量则关注任务之间是否存在清晰的先后依赖（20分）。若结果中存在字段缺失、任务过少或依赖关系表达不足等问题，系统会自动给出针对性的建议。结果评估功能前端展示如图 5.12 所示。



图 5.12 自动评估结果展示图

评估模块的实现不仅返回综合得分，还同时返回结构化完整度、用户故事质量、任务拆解质量和依赖关系质量四个维度的分项得分，以及针对字段缺失、任务过少、依赖不足等问题的改进建议。前端进一步将综合得分、维度评分和建议标签并列展示，使评估结果既能服务用户即时判断，也能作为第 6 章实验比较中的自动评分依据。这一功能使系统不仅具有生成能力，还具备一定的结果评价能力。对于毕设系统而言，这样的设计可以更好地体现系统在“生成”之外的“分析与

反馈”价值，也为后续第 6 章的实验比较和结果分析提供了依据。评估模块核心代码如表 5.6 所示。

表 5.6 结果评估核心代码表

#评分细则

```
def evaluate_result(result: Dict[str, Any]) -> Dict[str, Any]:
    structured = result.get("structured_requirement", {})
    stories = result.get("user_stories", [])
    tasks = result.get("tasks", [])
    structured_score = 0
    structured_score += 10 if structured.get("roles") else 0
    structured_score += 10 if structured.get("actions") else 0
    structured_score += 5 if structured.get("conditions") else 0
    structured_score += 10 if structured.get("goals") else 0
    task_score = 0
    if tasks:
        task_score += 15 if len(tasks) >= 2 else 5
    return {
        "total_score": structured_score + task_score,
        "summary": get_summary(structured_score + task_score)
    }

def get_summary(score: int) -> str:
    if score >= 85:
        return "结果较好，结构完整性和任务拆解质量较高。"
    if score >= 70:
        return "结果基本可用，但仍有一定优化空间。"
    if score >= 60:
        return "结果一般，建议进一步优化提示词和知识增强。"
    return "结果较弱，建议重新组织需求描述并完善知识库。"
```

5.2.7 相关代码辅助生成功能实现

在完成用户故事和任务拆解后，系统还支持围绕选中的某一条用户故事进一步生成相关代码骨架。用户在页面中选择目标用户故事后，可触发代码生成接口，系统会将原始需求、当前用户故事以及其对应的任务列表一并发送给后端，由模型输出项目结构建议、前端代码、后端代码、数据库 SQL 以及使用说明。代码生成结果展示如图 5.13 所示。



图 5.13 用户故事生成代码图

需要说明的是，该功能生成的是围绕某条故事的核心代码骨架，而非完整可运行项目。在当前版本中，代码辅助生成结果还附带代码生成耗时、模型耗时和生成状态等运行信息，并在前端以提示条形式展示。这样，代码辅助生成不再只是展示若干代码文本，而是能够明确说明该功能是否完成一次有效模型调用、耗时主要集中在哪一阶段，从而为第 6 章中的性能分析和功能可用性分析提供补充证据。其主要价值在于为后续开发提供接口设计、页面实现和数据库建模的初步参考。对于课程设计、原型验证和功能推演场景而言，这种基于用户故事的代码辅助生成能够显著缩短从需求分析到初步实现的过渡时间。相关代码生成核心代码如表 5.7 所示。

表 5.7 输出代码功能说明表

```
def generate_related_code_with_trace(requirement: str, story: str,
tasks: List[Dict[str, Any]]) -> Dict[str, Any]:
    prompt = build_code_prompt(requirement, story, tasks)

    response = client.chat.completions.create(
        model="deepseek-chat",
        messages=[{"role": "user", "content": prompt}],
        response_format={"type": "json_object"},
        temperature=0.2
    )

    content = response.choices[0].message.content
    parsed = json.loads(content)
    normalized = normalize_code_result(parsed)

    return {
        "result": normalized
    }
```

生成代码功能输出说明如表 5.8 所示。

表 5.8 输出代码功能说明表

输出项	内容说明	用途
项目结构	推荐的文件和目录结构	帮助理解实现边界
前端代码	与故事对应的前端界面骨架	支持页面原型实现
后端代码	与故事对应的接口逻辑骨架	支持服务端功能实现
数据库 SQL	与故事相关的数据表设计建议	支持数据建模
使用说明	对生成代码的补充解释	帮助后续修改与集成

5.2.8 知识模块管理与统计模块功能实现

为了保证系统在初始知识库之外具备持续扩展能力，本文在后端实现了知识模块管理与统计功能。该功能主要包括知识模块查询、知识模块新增和知识统计信息获取三部分。其中，知识模块查询接口用于返回当前数据库中全部领域知识

模块及其详细内容，便于系统维护人员查看已有知识库结构；知识模块新增接口用于将新的业务模块写入数据库，从而支持不同场景下知识库的持续扩充；知识统计接口则用于从整体层面对知识库规模、分类数量、别名数量以及各类规则条目数进行统计分析，为系统运行状态查看和后续论文实验说明提供支撑。

在具体实现中，系统基于 SQLite 和 SQLAlchemy 对知识模块进行持久化管理，并通过统一的数据转换逻辑将数据库中的文本化字段还原为前端可直接使用的结构化列表。对于新增知识模块请求，系统在接口层首先对模块名称进行非空校验，并检查当前数据库中是否已存在同名模块，若不存在，则将模块名称、分类、别名、必选要素、前置条件、异常场景、典型任务和安全约束等信息统一写入数据库。对于知识统计功能，系统则在后端对全部知识模块进行遍历汇总，统计当前知识库的模块总数、分类总数、别名数量以及规则信息分布情况，并以结构化格式返回统计结果。知识命中管理模块如图 5.14 所示。



图 5.14 知识命中管理图

该功能的实现使系统的知识库不再是固定不变的静态资源，而具备了一定的维护与扩展能力。一方面，系统能够通过知识模块管理功能支持后续新增业务场景，提升不同领域下的适用性；另一方面，知识统计功能能够帮助用户直观掌握当前知识库覆盖情况，为知识增强能力分析和实验结果说明提供依据。对于毕业设计原型系统而言，这种设计增强了系统的工程完整性，也为后续知识库扩展和维护优化提供了基础。

5.2.9 导出功能实现

系统还支持将当前页面中的知识增强内容、结构化需求、用户故事、任务清单、评估结果和代码生成结果统一导出为 Markdown 文本，在导出的 Markdown 结果中，系统还会同步写出知识命中模块、上下文来源以及运行耗时等证据信息，从而使实验记录不仅保留结果文本，也保留生成过程的关键观测指标。这种导出能力有利于后续论文附录整理、实验复核和演示留档，便于实验留档、过程记录。由此，系统最终形成了从需求输入到代码辅助输出的闭环实现路径。导出 Markdown 文本如图 5.15 所示。



图 5.15 导出 Markdown 展示图

5.3 系统集成过程

系统集成过程主要包括后端服务启动、数据库初始化、前端代理联调和容器化部署四个环节。在本地开发环境下，后端通过 FastAPI 和 uvicorn 启动服务，自动完成数据表创建和知识库初始化；前端通过 vue.config.js 中的代理配置，将浏览器中的/api 请求自动转发到本地后端服务地址，从而避免前后端分离开发中的跨域和地址硬编码问题。

完成系统集成后，用户在浏览器中输入需求或代码片段，即可触发后端生成链路，并在前端连续查看知识增强结果、用户故事、任务清单、依赖关系图和评

估信息。当用户选中某条用户故事后，系统还能继续生成对应的代码骨架，实现从需求输入到代码辅助输出的端到端闭环。这表明系统已具备较完整的联调与部署能力。系统集成关键配置文件说明如表 5.9 所示。

表 5.9 系统集成关键配置说明表

配置文件	关键配置项	主要作用
.env.example	DEEPSEEK_API_KEY、 DATABASE_URL	提供后端环境变量模板
.env.example	VUE_APP_API_BASE_URL	支持前端接口地址覆盖
vue.config.js	/api 代理到本地后端	支持开发环境前后端联调
docker-compose.yml	backend、frontend、backend-storage	实现前后端容器编排与数据持久化
Dockerfile	安装依赖并启动 Uvicorn	构建后端镜像
Dockerfile	构建前端并交由 Nginx 托管	构建前端镜像
nginx.conf	/api 反向代理到 backend:8000	提供统一访问入口

5.4 本章小结

本章围绕系统实现过程，对数据层初始化、核心业务层构建、前端交互实现、各功能模块落地以及系统集成方式进行了说明。通过 SQLite 知识库、FastAPI 服务、Vue 前端界面和 Docker 部署方案的协同，系统已经能够完成知识增强、用户故事生成、任务拆解、依赖可视化、结果评估和代码辅助生成等核心功能，验证了系统设计方案的可实现性和工程可落地性。

第 6 章 系统测试与结果分析

6.1 测试环境

系统测试的目的在于验证智能用户故事生成系统是否能够稳定、正确地完成需求输入、知识增强、用户故事生成、任务拆解、结果评估和代码辅助生成等核心功能，并进一步检验系统在模块协同、页面联调和整体运行过程中的可用性与可靠性。结合本系统的功能特点，本章测试从单元测试、集成测试、系统测试、功能测试、性能测试和验收测试六个层面展开，既覆盖后端核心逻辑，也覆盖前后端联调和用户使用流程。

本章测试采用自动化测试与人工测试相结合的方法。后端测试主要基于接口调用与函数级校验完成，重点验证知识增强、结构化生成和结果评估链路是否正确；前端测试主要结合页面构建验证和功能联调进行，重点考察页面展示、交互联动与结果导出能力；性能测试则通过多次重复调用关键接口记录响应时间，从而评估系统在当前环境下的运行效率。

当前验证中，后端应用可正常导入并启动测试客户端，健康检查、知识增强、知识库查询、生成与评估、代码生成等关键接口均可访问；前端构建在当前环境下执行成功，并成功生成 `dist` 目录，说明系统具备基本的可部署性。系统测试环境如表 6.1 所示。

表 6.1 系统测试环境表

测试项	环境或工具	说明
操作系统	Windows	本地开发与测试环境
后端框架	FastAPI	提供系统核心接口服务
后端运行方式	Uvicorn/TestClient	分别用于手动运行与自动化接口测试
前端框架	Vue 2	构建单页面交互界面
前端组件	Element UI、ECharts	分别用于界面展示与依赖图可视化
数据存储	SQLite	存储知识库模块数据
模型服务	DeepSeek API	提供用户故事生成与代码生成能力
测试方法	自动化测试+人工测试	同时覆盖后端逻辑和页面交互

6.2 测试用例设计

为了保证测试结果具有代表性和可信度，本章测试用例设计遵循以下原则：一是覆盖系统主要功能链路，包括需求输入、知识增强、结构化生成、任务拆解、结果评估和代码生成；二是覆盖异常与边界场景，例如空输入、缺失字段和异常返回；三是区分不同测试层次，避免单元测试、集成测试和系统测试内容重复；四是采用可复现的测试输入与明确的预期输出，以保证测试结果具有可比较性。

在测试过程上，首先通过单元测试验证基础函数行为是否正确；随后通过集成测试检查接口之间的数据传递和模块协同是否稳定；再通过系统测试验证核心业务链路能否从输入一直执行到结果输出；在此基础上进行面向用户的功能测试、性能测试和验收测试，形成完整的测试闭环。测试用例总体设计如表 6.2 所示。

表 6.2 测试用例总体设计表

测试类型	测试对象	测试重点	预期结果
单元测试	文本归一化、关键词命中、结果规范化、评估函数	验证基础逻辑正确性	函数返回结果符合预期
集成测试	健康检查、知识增强、知识库查询、生成与评估、代码生成接口	验证模块之间协同调用	接口返回成功且数据结构完整
系统测试	完整业务链路	验证从需求输入到结果输出的闭环	生成故事、任务、评分和代码骨架
功能测试	前端页面功能	验证页面交互和结果展示	页面可正常输入、显示、导出
性能测试	关键接口响应时间	验证系统在当前环境下的响应效率	响应时间满足基本使用需求
验收测试	毕设目标对应功能	验证系统是否达到预期目标	核心功能均可完成

6.3 单元测试

单元测试则主要瞄准系统内部那些相对独立的函数，重点验证文本预处理、知识匹配、模型结果规范化和评估计算这些底层逻辑是否正确，相较于接口测试，它更关注函数本身的输入输出对应关系，能在更小粒度上发现潜在问题。这次单元测试选取了文本归一化函数、关键词命中函数、结果规范化函数和评估函数作为测试对象，结果表明它们在典型输入下都能返回预期结果；除了常规的函数验

证，还通过模拟模型返回非法 JSON 和模拟推理异常两种方式，对生成模块的异常兜底能力进行了单元级校验。测试表明，当模型返回不合法内容时，系统可以正确识别 `invalid_json` 这个错误类型；推理阶段发生异常时，系统则能回退到标准错误结构并标记 `inference_error`，从而避免前端因异常结果直接失效，这反映出系统底层逻辑具备较好的稳定性，也为上层接口调用打下了可靠的基础。单元测试用例如表 6.3 所示。

表 6.3 单元测试用例表

用例编号	测试对象	测试输入	预期结果	实际结果	是否通过
UT-01	文本归一化	“登录注册类 ”	去除空白和无关后缀，得到“登录注册”	返回“登录注册”	通过
UT-02	关键词命中	“用户可以注册并登录系统”，	能正确识别命中关系	返回 True	通过
UT-03	结果规范化	结构化需求、用户故事和任务	返回字段完整、任务类型合法的规范化结果	规范化成功	通过
UT-04	评估函数	规范化后的生成结果对象	返回总分、维度和建议	成功返回 <code>total_score</code> 、 <code>dimension_scores</code> 、 <code>suggestions</code>	通过

6.4 集成测试

集成测试主要是为了检验后端各模块能不能很好地协同工作，重点考察接口层、知识增强模块、生成模块、评估模块和代码生成模块之间数据传递的准确性以及返回结构的统一性，跟单元测试比起来，它更贴近真实的运行流程，能反映出模块组合之后的稳定性。这次集成测试采用了接口自动化方式，依次调用了健康检查、知识增强预览、知识库模块查询、生成并评估、代码生成和异常输入处理等接口，从得到的结果看，系统接口在正常输入下都能返回合法结果，碰到异常输入也能给出清晰的错误提示。在异常接口联调这块，还进一步验证了空需求输入、知识库未命中、空代码输入和代码生成缺失用户故事等场景，结果表明系统都能返回明确的错误消息或回退上下文，没出现接口崩溃、前端空白页或结果结构不一致这类问题，这足以看出异常输入处理机制已从函数级校验扩展到了接

口级联调阶段。其中，知识库查询接口成功返回了 38 个知识模块，生成并评估接口和代码生成接口也都顺利返回了业务数据，说明后端核心链路已经具备了较好的协同能力。集成测试用例如表 6.4 所示。

表 6.4 集成测试用例表

用例编号	接口名称	测试内容	预期结果	实际结果	是否通过
IT-02	知识增强预览接口	输入“用户可以注册并登录系统查看个人信息”	返回知识增强上下文	返回成功	通过
IT-03	知识库查询接口	查询全部知识模块	返回知识模块总数及列表	返回 total=38	通过
IT-04	生成并评估接口	输入正常业务需求	返回故事、任务和评估结果	返回成功	通过
IT-05	代码生成接口	选择一条用户故事	返回项目结构和代码骨架	返回成功	通过
IT-06	空需求异常处理	需求文本为空	返回错误提示“需求文本不能为空”	返回错误信息	通过
IT-07	空代码异常处理	代码文本为空	返回错误提示“代码文本不能为空”	返回错误信息	通过
IT-01	健康检查接口	检查服务运行状态	返回 success=true 和服务状态信息	返回正常	通过

6.5 系统测试

系统测试关注的是完整业务链路是否能够顺利运行。与集成测试不同，系统测试不再单独关注某个接口或某个函数，而是从用户视角出发，验证系统能否完成从需求输入、知识增强、用户故事生成、任务拆解、结果评估到代码辅助生成的端到端流程。

在本次系统测试中，以“用户可以注册登录系统，并查看和修改个人资料”为输入样例，系统首先生成知识增强上下文，随后返回结构化需求、4 条用户故事、13 条任务清单以及评估结果，其中综合得分为 100 分。在用户选中某条故事后，系统继续生成了项目结构建议，首次返回 4 条项目结构路径。测试结果说明，系统能够顺畅地完成主要业务闭环，满足从需求分析到工程辅助输出的整体设计目标。系统测试用例如表 6.5 所示。

表 6.5 系统测试用例表

用例编号	测试场景	测试过程	预期结果	实际结果	是否通过
ST-01	需求生成完整链路	输入自然语言需求，调用生成并评估接口	返回知识上下文、故事、任务和评估	返回 4 条故事、13 条任务、总分 100	通过
ST-02	用户故事代码生成链路	在已有结果上选择故事并生成代码	返回项目结构和代码骨架	返回项目结构建议	通过
ST-03	端到端流程闭环	从输入到生成再到代码输出	流程连续、无中断	流程执行成功	通过

端到端运行效果如图 6.1 所示。



图 6.1 端到端运行效果图

6.6 功能测试

功能测试主要从最终用户使用系统的角度出发，验证页面功能是否齐全、操作流程是否顺畅、结果展示是否符合预期。与系统测试相比，功能测试更强调“用户能否用起来”，重点考察页面交互行为和可见功能是否符合设计要求。

本系统功能测试主要包括自然语言需求生成、代码片段生成故事、用户故事选择联动、任务筛选与搜索、任务依赖图展示、结果导出和页面重置等内容。从当前实现与联调方式看，系统已经具备上述主要功能；此外，前端项目在当前环境下能够成功构建，并生成 `dist` 目录，系统页面端具备基本可部署性。功能测试用例如表 6.6 所示。

表 6.6 功能测试用例表

用例编号	功能项	测试内容	预期结果	实际结果	是否通过
FT-01	自然语言需求生成	输入需求并点击生成	页面展示知识上下文、故事、任务和评估	功能满足设计要求	通过
FT-02	代码片段生成故事	输入代码片段并生成	页面返回提取后的需求及故事结果	功能满足设计要求	通过
FT-03	用户故事联动	点击故事卡片或任务节点	当前选中故事同步更新	功能满足设计要求	通过
FT-04	任务筛选与搜索	按任务类型或关键字筛选	表格结果正确变化	功能满足设计要求	通过
FT-05	任务依赖图展示	查看依赖图并点击节点	节点展示正确且支持联动	功能满足设计要求	通过
FT-06	结构化结果	查看生成后的结构化需求	生成对应的结构化结果	功能满足设计要求	通过
FT-07	结果评估	检查结果评估模块评分	根据生成结果生成对应评分	功能满足设计要求	通过
FT-08	相关代码生成	点击一键生成相关代码	成功生成相关代码骨架	功能满足设计要求	通过
FT-09	结果导出	点击导出 Markdown	当前结果导出为 Markdown 文件	功能满足设计要求	通过
FT-10	页面重置	点击重置页面	输入和结果状态全部清空	功能满足设计要求	通过
FT-11	前端构建	执行前端打包构建	成功生成 dist 目录	构建成功	通过

6.7 性能测试

性能测试用于评估系统在当前运行环境下的响应效率和稳定性。考虑到本系统既包含本地处理接口，也包含依赖外部大语言模型服务的生成接口，纯 LLM 与直接和 AI 对话在效果上难以区分，因此本次测试重点考察 LLM+规则模板、LLM+知识库增强两种策略下的结果：一类是本地轻量接口测试，采用 20 次重复调用统计平均耗时与最大耗时；另一类是核心生成链路测试，采用 10 次重复调用记录平均耗时与最大耗时。

本次测试采用 Python 自动化测试脚本对系统接口进行性能统计，使用 FastAPI 提供的 TestClient 模拟接口请求，并利用 Python 标准库中的高精度计时函数 time.perf_counter 记录请求开始到响应返回的时间，最终对多次测试结果进行平均值和最大值统计分析，该方法适用于单机环境下的接口响应性能测试，能够较准

确反映系统在当前实验环境中的处理效率。由于测试环境受限，本次测试不考虑并发因素造成的性能影响。性能测试用例如表 6.7 所示。

表 6.7 性能测试用例表

策略	接口名称	调用次数	平均耗时	最大耗时	结果说明
LLM+规则模板	/health	20	2.16ms	2.77ms	本地轻量接口，响应稳定
LLM+规则模板	/preview_context	20	4.63ms	5.26ms	无知识增强
LLM+规则模板	/knowledge_modules	20	5.11ms	5.98ms	无知识增强
LLM+规则模板	/generate_and_evaluate	10	6.16s	6.55s	受模型推理影响
LLM+规则模板	/generate_code	10	12.07s	14.24s	耗时最高
LLM+知识库增强	/health	20	2.08ms	2.81ms	响应稳定
LLM+知识库增强	/preview_context	20	4.99ms	5.62ms	知识增强预览响应较快
LLM+知识库增强	/knowledge_modules	20	6.54ms	6.98ms	知识库查询稳定
LLM+知识库增强	/generate_and_evaluate	10	5.83s	6.21s	受模型推理影响
LLM+知识库增强	/generate_code	10	12.91s	16.84s	代码生成接口耗时最高，但结果稳定

测试结果表明，LLM+规则模板情况下，健康检查、知识增强预览和知识库查询等本地接口的响应速度较快，平均响应时间均低于 7ms；而 generate_and_evaluate 和 generate_code 两个依赖模型服务的核心接口平均响应时间分别为 6.16s 和 12.07s，最大响应时间分别为 6.55s 和 14.24s；LLM+知识库增强情况下，generate_and_evaluate 和 generate_code 两个依赖模型服务的核心接口平均响应时间分别为 5.83s 和 12.91s，最大响应时间分别为 6.21s 和 16.84s。在提升生成质量的同时，知识增强不仅没有显著增加系统响应时间，反而在核心用户故事生成环节使平均耗时缩短，仅在代码生成环节出现了可接受的小幅增加，主要是由于生成的代码包含了更多的业务约束和安全检查。整体来看，系统性能瓶颈主要来自外部大语言模型的推理过程，知识增强在提升生成质量的同时，没有显著增加系统的响应时间。为能直观展示性能测试证据，系统前端添加了运行耗时证据，具体展示如图 6.2 所示。

运行耗时证据



图 6.2 运行耗时展示图

6.8 验收测试

验收测试的目标是从课题任务完成度的角度，对系统整体功能进行最终确认。与前述测试不同，验收测试不再关注局部函数或单个接口，而是围绕毕设目标检查系统是否已经具备可交付的核心能力。

结合本文目标，验收测试重点围绕五个方面展开：是否支持自然语言需求输入与知识增强，是否能够生成标准用户故事，是否能够拆解出可执行任务并构建依赖关系，是否能够给出评估结果，以及是否能够继续输出相关代码骨架。从当前测试结果看，上述核心目标均已满足，说明系统已经达到毕业设计的基本验收要求。系统验收测试如表 6.8 所示。

表 6.8 验收测试表

验收项	验收标准	验收结果	是否通过
需求输入与知识增强	能接收需求并返回知识增强上下文	已满足	通过
用户故事生成	能生成符合模板的用户故事	已满足	通过
任务拆解与依赖构建	能生成任务清单并表达依赖关系	已满足	通过
结果评估	能返回得分、结论和建议	已满足	通过
代码辅助生成	能生成项目结构和代码骨架	已满足	通过
页面构建与展示	前端可构建并可展示结果	已满足	通过

6.9 测试结果

综合各项测试结果可以看出，本系统在当前阶段已经完成了预期的主要功能验证。单元测试表明基础函数逻辑正确；集成测试表明各接口之间能够稳定协同；系统测试表明从需求输入到代码辅助输出的完整链路能够成功运行；功能测试表明页面主要交互功能具备可用性；性能测试表明本地接口响应较快，核心生成链路在外部模型参与下仍保持可接受的响应时间；验收测试则进一步说明系统整体功能已经达到毕设任务要求。这些结果说明系统在正确性、完整性和稳定性三个方面均取得了较好的测试表现。测试结果汇总如表 6.9 所示。

表 6.9 测试结果汇总表

测试类型	核心结果	结论
单元测试	4 项基础函数测试均通过	底层逻辑正确
集成测试	7 项接口测试均通过，知识模块 total=38	模块协同稳定
系统测试	生成 4 条故事、13 条任务、总分 100	核心业务闭环成立
功能测试	页面主要功能完整，前端构建成功	用户侧功能可用
性能测试	本地接口小于 6ms，核心生成链路 6.16s 和 12.07s	性能满足演示和基本使用需求
验收测试	课题目标对应功能均满足	达到验收要求

为全面反映系统在测试阶段暴露出的不足与异常情况，本文对测试过程中出现的典型失败案例进行了整理与归纳，具体内容如表 6.10 所示。

表 6.10 测试失败汇总表

样本编号	异常类型	系统表现	原因分析	改进建议
F01	空输入	拦截并返回“需求文本不能为空”	输入校验触发	保持现状
F02	知识库未命中	回退到通用软件工程上下文，miss_reason 为 no_domain_match	样本超出知识库覆盖范围	扩充行业知识模块
F03	歧义输入	生成泛化故事，评估提示信息不足	原始需求不明确	增加追问式交互
F04	冲突约束	需要人工评审识别逻辑冲突	模型难以稳定解决矛盾约束	增加规则冲突检测
F05	空代码输入	直接返回“代码文本不能为空”	输入校验触发	保持现状
F06	缺失用户故事的代码生成	直接返回“用户故事不能为空”	接口前置校验触发	保持现状

6.10 测试结论与分析

根据测试的情况来看，这套系统整体运行得比较稳定，能较好地满足第 2 章里提到的那些功能需求与非功能需求。一是在正确性这个维度上，系统里的基础函数、接口返回的数据结构还有端到端的业务链路，都已经顺利通过了验证，这能说明系统在需求输入、知识增强、故事生成、任务拆解和结果评估这些关键环节中，没藏着什么明显的逻辑错误；二是在完整性上，它不光能生成用户故事，还可以把任务依赖关系和代码骨架一并输出出来，表现出了挺强的功能闭环能力；三是在可用性方面，前端页面支持了好几种输入方式和结果展示形式，后端的接口也有了一套统一的错误处理机制，能应对日常的基本使用场景。

不过也得看到，系统性能这块出现了一个挺明显的反差：本地处理的速度相当快，一到模型生成那里就慢下来了。本地接口花费的时间很少，说明知识检索和数据库访问算不上拖慢速度的瓶颈；核心生成接口和代码生成接口的耗时就要高出不少了，这主要是外部大语言模型的推理消耗与网络环境造成的。这样一种表现和当下常见的大模型应用系统的特点是吻合的，往后做优化的时候，可以考虑从缩短提示词长度、增加结果缓存、压减模型调用次数，还有引入异步处理这些方向上去做改进。

把这些结果综合起来看，能得出这样一个结论：当前这套系统已经拥有了比较完整的需求生成和分析方面的能力，在知识增强的支撑下，做到了用户故事生成、任务拆解、依赖表达、结果评估和代码辅助输出，整体上达到了系统设计的预期目标，测试结果的合理性和可信度也有一定支撑。

6.11 本章小结

本章围绕智能用户故事生成系统的运行效果，从测试环境、测试用例设计、单元测试、集成测试、系统测试、功能测试、性能测试和验收测试等方面对系统进行了较为系统的检验，并给出了测试结果、测试分析与结论。测试结果表明，系统在正确性、稳定性和可用性方面均表现良好，能够较好地满足课题目标和系统需求。上述测试结论也为后续第 7 章的全文总结与不足分析提供了支撑。

第 7 章 结论与展望

7.1 研究成果总结

这篇论文围绕“知识融入大语言模型的智能用户故事生成系统”这个主题，针对传统需求分析与用户故事编写里头一直存在的几个问题，比如需求表达不够规范、领域知识过多依赖个人经验、任务拆解做得不到位、以及输出的结果很难直接拿来指导开发实施等，开展了从系统需求分析、关键技术研究，到系统设计、实现与测试在内的一连串工作，最终形成了一套相对完整的智能用户故事生成方案。

在需求分析的环节，论文从业务需求、用户需求、功能需求、非功能需求以及约束需求这几个维度，对系统的建设目标和功能边界做了系统的梳理，把系统要具备的自然语言需求输入、代码片段反向分析、知识增强、用户故事生成、任务拆解、依赖关系构建、结果评估还有代码辅助输出这些核心能力都给明确了下来，这部分工作为后面的系统设计和实现提供了一个清楚的依据。

在关键技术研究上，论文分析了大语言模型在软件需求工程里头的应用价值和局限，点出了只靠通用模型的话，很难稳定地满足业务完整性和结构规范性方面的要求。为了解决掉这个问题，论文引入了一个轻量级的领域知识库，再把知识增强检索机制和提示词工程结合到一块，搭建起了一套“知识+模型”的协同生成方法，让系统在生成需求的过程中，有能力去补充前置条件、异常场景、典型任务和安全约束之类的业务信息，这样一来，生成结果的完整性和可用性就得到了提高。

在系统设计与实现的阶段，论文完成了基于 FastAPI 的后端服务与基于 Vue 的前端交互界面的开发工作，构建出了从需求输入一直到结果展示的一个完整业务闭环。当前这个系统已经能支持的功能有不少，像自然语言需求生成、通过代码片段来生成用户故事、知识增强上下文的展示、结构化需求提取、用户故事卡片的展示、任务清单的展示、任务依赖图的可视化、结果评估、相关代码的生成，还有 Markdown 导出这些，全都包括了。其中，知识库已经覆盖了几个典型的软件业务场景，能够给系统生成提供基本的先验知识支撑。

在测试与结果分析的层面，论文从单元测试、集成测试、系统测试、功能测试、性能测试，再到验收测试等多个层次，对系统进行了验证。从得到的测试结果来看，系统已经可以比较稳定地完成知识增强、用户故事生成、任务拆解、结果评估和代码辅助生成这一系列的核心流程，前后端的联调和前端的构建也都能顺顺利利地完成，整体的实现效果已经达到了毕业设计的预期目标。

综合来看，这篇论文取得的主要成果，可以从几个方面来概括。第一方面，是构建出了一套面向用户故事生成场景的智能需求处理方法，把大语言模型的能力跟轻量级领域知识库有机地结合到了一起。第二方面，是设计并实现了一个从原始需求输入，到任务拆解，再到代码辅助输出的系统化平台，这让生成的结果能更加贴近软件工程的实践。第三方面，是通过多层次的测试，验证了这套系统方案的可行性，为以后在更复杂场景里进行扩展和应用，打下了一个基础。

7.2 研究局限与未来展望

虽然本次研究已经开发出一种结合大语言模型以及轻量级知识库的智能用户故事生成系统，在语义理解、结构化转换以及任务推理上取得一定成果，但是仍然存在一些问题。一方面，系统所使用的领域知识库规模较小，仅针对常见业务场景进行建模，对于新行业或者高度个性化行业适应性较差。另一方面，大语言模型对于模糊、歧义或者非常抽象的需求描述，容易产生错误或者幻觉，导致生成的结果不够准确。另外，目前系统对于多次交互的需求细化的支持还不够完善，用户的反馈方式比较简单，不能很好地根据用户的反馈来调整知识推理的过程。下一步的工作主要包括三个方向：一是扩大知识库的规模以及深度，加入动态的知识更新以及跨领域的迁移学习的方法，提高系统的鲁棒性。二是利用强化学习以及人类偏好对齐的方法，在多次推理过程中保持决策的一致性。三是加强人机协作的能力，让用户可以通过自然语言对生成的内容进行修改并且可以自动反馈给知识库，形成一个闭环。根据中国信息通信研究院 2025 年的数据，敏捷开发在国内软件行业中的应用范围不断扩大，与此同时，企业对于智能化需求分析与自动化需求处理工具的关注度也持续提升，该系统有希望在此基础上进一步发展，为更多软件工程项目提供服务。

参考文献

- [1] Touvron H, Lavril T, Izacard G, et al. LLaMA: Open and Efficient Foundation Language Models[EB/OL].(2023-02-27)[2026-05-11].
<https://arxiv.org/abs/2302.13971>.
- [2] Storey M A, Zagalsky A, Figueira Filho F, et al. How social and communication channels shape and challenge a participatory culture in software development[J]. IEEE Transactions on Software Engineering, 2016, 43(2): 185-204.
- [3] Nuseibeh B, Easterbrook S. Requirements engineering: a roadmap[C]// Proceedings of the Conference on the Future of Software Engineering. 2000: 35-46.
- [4] Hemmat A, Sharbaf M, Kolahdouz-Rahimi S, et al. Research directions for using LLM in software requirement engineering: a systematic review[J]. Frontiers in Computer Science, 2025, 7: 1519437.
- [5] Lubos S, Felfernig A, Tran T N T, et al. Leveraging LLMs for the quality assurance of software requirements[C]// 2024 IEEE 32nd International Requirements Engineering Conference (RE). IEEE, 2024: 389-397.
- [6] Lucassen G, Dalpiaz F, van der Werf J M E M, et al. Improving agile requirements: the quality user story framework and tool[J]. Requirements Engineering, 2016, 21(3): 383-403.
- [7] Inayat I, Salim S S, Marczak S, et al. A systematic literature review on agile requirements engineering practices and challenges[J]. Computers in Human Behavior, 2015, 51: 915-929.
- [8] 栗元邦, 彭蓉, 季晶晶, 等. 经验研究中情景感知需求获取与建模系统文献综述[J]. 软件学报, 2017, 29(2): 320-339.
- [9] Schön E-M, Thomaschewski J, Escalona M J. Agile Requirements engineering: a systematic literature review[J]. Computer Standards & Interfaces, 2017, 49: 79-91.
- [10] Hoy Z, Xu M. Agile software requirements engineering challenges-solutions—a conceptual framework from systematic literature review[J]. Information, 2023, 14(6): 322.
- [11] dos Santos C A, Bouchard K, Minetto Napoleão B. Automatic user story generation: a comprehensive systematic literature review[J]. International Journal of Data Science and Analytics, 2025, 20(1): 1-24.

- [12] Rahman T, Zhu Y. Automated User Story Generation with Test Case Specification Using Large Language Model[EB/OL].(2024-04-02)[2026-05-11].
<https://arxiv.org/abs/2404.01558>.
- [13] Ellsel C, Stark R. Advancing Requirements Engineering with Large Language Models[J]. Procedia CIRP, 2025, 136: 701-706.
- [14] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks [C]// Proc 34th Int Conf Neural Inf Process Syst. Red Hook: Curran Associates Inc, 2020: 1-16.
- [15] Gao Y, Xiong Y, Gao X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey[EB/OL].(2023-12-18)[2026-05-11].
<https://arxiv.org/abs/2312.10997>.
- [16] Peng B, Zhu Y, Liu Y, et al. Graph retrieval-augmented generation: a survey[J]. ACM Transactions on Information Systems, 2025, 44(2): 1-52.
- [17] Yang W, Some L, Bain M, et al. A comprehensive survey on integrating large language models with knowledge-based methods[J]. Knowledge-Based Systems, 2025, 318: 113503.
- [18] Pan S, Luo L, Wang Y, et al. Unifying large language models and knowledge graphs: a roadmap[J]. IEEE Transactions on Knowledge and Data Engineering, 2024, 36(7): 3580-3599.
- [19] Ghorbian M, Ghobaei-Arani M, Shakarami A. Large language models for code generation: A survey[J]. Computer Standards & Interfaces, 2026, 98: 104-165.
- [20] Qian C, Liu W, Liu H, et al. ChatDev: communicative agents for software development[C]// Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024: 15174-15186.
- [21]Antaputra A M M, Yong L T. A Systematic Literature Review on Agile Requirements Engineering and Artificial Intelligence[C]// 2024 22nd International Conference on ICT and Knowledge Engineering (ICT&KE). 2024: 1-6.
- [22] Hou X, Zhao Y, Liu Y, et al. Large language models for software engineering: a systematic literature review[J]. ACM Transactions on Software Engineering and Methodology, 2024, 33(8): 1-79.
- [23] Ferrari A, Spagnolo G O, Gnesi S. Pure: a dataset of public requirements documents[C]// 2017 IEEE 25th International Requirements Engineering Conference (RE). IEEE, 2017: 502-505.

-
- [24] Norheim J J, Rebentisch E. Structuring natural language requirements with large language models[C]// 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW). IEEE, 2024: 68-71.
- [25] Mokos K, Nestoridis T, Katsaros P, et al. Semantic modeling and analysis of natural language system requirements[J]. IEEE Access, 2022, 10: 84094-84119.
- [26] Yamani A, Baslyman M, Ahmed M. Leveraging LLMs for user stories in AI systems: UStAI dataset[C]// Proceedings of the 21st International Conference on Predictive Models and Data Analytics in Software Engineering. ACM, 2025: 21-30.
- [27] Zhang Z, Rayhan M, Herda T, et al. LLM-based agents for automating the enhancement of user story quality: an early report[C]// International Conference on Agile Software Development. Springer, 2024: 117-126.
- [28] Dente F, Dalpiaz F, Papotti P. Text2Stories: Evaluating the Alignment Between Stakeholder Interviews and Generated User Stories[EB/OL].(2025-10-08)[2026-05-12]. <https://arxiv.org/abs/2510.08622>.
- [29] Abed O, Nebe K, Abdellatif A B. AI-Generated user stories supporting Human-Centred development: an investigation on quality[C]// Communications in Computer and Information Science. 2024: 3-13.
- [30] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]. Advances in Neural Information Processing Systems, 2022, 35: 24824-24837.
- [31] Yao S, Yu D, Zhao J, et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models[EB/OL].(2023-05-17)[2026-05-12]. <https://arxiv.org/abs/2305.10601>.
- [32] Sahoo P, Singh A K, Saha S, et al. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications[EB/OL]. (2024-02-12)[2026-05-12]. <https://arxiv.org/abs/2402.07927>.
- [33] Ronanki K, Arvidsson S, Axell J. Prompt Engineering Guidelines for Using Large Language Models in Requirements Engineering[C]// Lecture Notes in Computer Science. 2026: 245-262.
- [34] Borgeaud S, Mensch A, Hoffmann J, et al. Improving language models by retrieving from trillions of tokens[C]// International Conference on Machine Learning. PMLR, 2022: 2206-2240.

- [35] Li Y, Li Z, Wang P, et al. A Survey of Graph Meets Large Language Model: Progress and Future Directions[EB/OL].(2023-11-21)[2026-05-12]. <https://arxiv.org/abs/2311.12399>.
- [36] Chen M, Tworek J, Jun H, et al. Evaluating Large Language Models Trained on Code[EB/OL].(2021-07-07)[2026-05-12]. <https://arxiv.org/abs/2107.03374>.
- [37] Rozière B, Gehring J, Gloeckle F, et al. Code Llama: Open Foundation Models for Code[EB/OL].(2023-08-24)[2026-05-12]. <https://arxiv.org/abs/2308.12950>.
- [38] Zan D, Chen B, Zhang F, et al. Large language models meet NL2Code: a survey[C]// Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2023: 7443-7464.
- [39] Barke S, James M B, Polikarpova N. Grounded Copilot: How Programmers Interact with Code-Generating Models[J]. Proc ACM Program Lang, 2023, 7(OOPSLA1): 78. DOI:10.1145/3586030.
- [40] Vaithilingam P, Zhang T, Glassman E L. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models[C]//Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems. New York: Association for Computing Machinery, 2022: 332. DOI:10.1145/3491101.3519665.
- [41] Wang J, Huang Y, Chen C, et al. Software Testing with Large Language Models: Survey, Landscape, and Vision[EB/OL].(2023-07-13)[2026-05-12]. <https://arxiv.org/abs/2307.07221>.
- [42] Li Y, Liu P, Wang H, et al, Wong W E. Evaluating large language models for software testing[J]. Computer Standards & Interfaces, 2025, 93: 103942.
- [43] Tong W, Zhang T. CodeJudge: Evaluating Code Generation with Large Language Models[EB/OL].(2024-10-03)[2026-05-12]. <https://arxiv.org/abs/2410.02184>.
- [44] 陈慧敏, 刘知远, 孙茂松. 大语言模型时代的社会机遇与挑战[J]. 计算机研究与发展, 2024, 61(05): 1094-1103.
- [45] 王昀, Hu M, Ta N, 等. 大语言模型及其在政务领域的应用[J]. 清华大学学报(自然科学版), 2024, 64(04): 649-658.
- [46] 孙毅, 裘杭萍, 郑雨, 等. 自然语言预训练模型知识增强方法综述[J]. 中文信息学报, 2021, 35(07): 10-29.

- [47] 张鹤译, 王鑫, 韩立帆, 等. 大语言模型融合知识图谱的问答系统研究[J]. 计算机科学与探索, 2023, 17(10): 2377-2388.
- [48] Feng Z, Zhu K, Ma W, et al. 浅谈大模型时代下的检索增强: 发展趋势, 挑战与展望 (Enhancing large language models with retrieval-augmented techniques: trends, challenges, and prospects)[C]// Proceedings of the 23rd Chinese National Conference on Computational Linguistics (Volume 2: Frontier Forum). 2024: 151-168.
- [49] 王亮. 检索增强生成(RAG)驱动的知识服务: 原理、范式及评估[J]. 科技与出版, 2025(04): 37-46.
- [50] 王春晖, 金芝, 赵海燕, 等. 一种用户故事需求质量提升方法[J]. 计算机研究与发展, 2021, 58(04): 731-748.
- [51] 荣国平, 刘天宇, 谢明娟, 等. 嵌入式系统开发中敏捷方法的应用研究综述[J]. 软件学报, 2014, 25(02): 267-283.
- [52] Project Management Institute, Agile Alliance. 敏捷实践指南[M]. 北京: 电子工业出版社, 2018: 7-10.
- [53] 王春晖, 金芝, 赵海燕, 等. 人机协作的用户故事场景提取与迭代演进[J]. 软件学报, 2019, 30(10): 3186-3205.
- [54] Ronanki K, Cabrero-Daniel B, Berger C. ChatGPT as a tool for user story quality evaluation: trustworthy out of the box?[C]// International Conference on Agile Software Development. Cham: Springer Nature Switzerland, 2022: 173-181.
- [55] Santos R, Steinmacher I, Conte T, et al. AI-Generated user stories: are they good enough?[C]// Simpósio Brasileiro de Engenharia de Software (SBES). SBC, 2025: 741-747.
- [56] White J, Hays S, Fu Q, et al. Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design[M]// Generative ai for effective software development. Cham: Springer Nature Switzerland, 2024: 71-108.
- [57] White J, Fu Q, Hays S, et al. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT[EB/OL]. (2023-02-22)[2026-05-12]. <https://arxiv.org/abs/2302.11382>.

- [58] Shrivastava D, Larochelle H, Tarlow D. Repository-level prompt generation for large language models of code[C]// Int Conf Mach Learn. PMLR, 2023: 31693-31715.
- [59] Du X, Liu M, Wang K, et al. Evaluating large language models in class-level code generation [C] // Proc IEEE/ACM 46th Int Conf Softw Eng. New York: Association for Computing Machinery, 2024: 1-13.
- [60] Yeo S, Ma Y S, Kim S C, et al. Framework for evaluating code generation ability of large language models[J]. ETRI Journal, 2024, 46(1): 106-117.

致谢

时光荏苒，四年的本科生涯即将画上句号。回顾这段求学之路，从最初对大学生活的懵懂好奇，到如今能够独立完成毕业设计，每一步都离不开身边人的支持与帮助。

首先，我要衷心感谢我的指导老师黄江平。从选题的确立、技术方案的选型，到代码实现中遇到瓶颈时的耐心点拨，再到论文撰写过程中的逐字逐句审阅，老师始终以严谨的治学态度和渊博的专业知识为我指引方向。特别是在项目架构设计和功能优化方面，老师提出的宝贵建议让我对软件工程的方法论有了更深刻的理解。

同时感谢与我并肩而行的舍友，感谢你们四年来的帮助和宽容，见证了彼此从一个学生转变为一个成年人，还要感谢远方的挚友们，在生活中不断鼓舞着我，在你们的实际经历中明确了未来的方向。希望我们都有伟大前程。

最后感谢我的家人，倾尽所有托举，让我窥见更广阔的天地，虽然最后还是回到了家乡的央企工作，但他们依旧无条件支持我，做我最安稳的避风港，亦是我的坚强后盾。

落笔千山，最后是自己，跌跌撞撞向阳成长，历经迷茫仍拨开云雾见光明。

毕业不是终点，而是新的起点。我将带着在大学里收获的知识、友谊与感动，在未来的道路上继续前行。